جامعة مولاي إسماعيل
UNIVERSITÉ MOULAY ISMAÏL

كلية العلوم القانونية و الاقتصادية و الاجتماعية
FACULTÉ DES SCIENCES JURIDIQUES ECONOMIQUES ET SOCIALES

Finance and Data Science

Banking Fraud Detection Using Isolation Forest

Final Project Report

Prepared by:
Mir Amina
Naffah Oumaima

Supervised by:
Mr Alj Abdelkamel

Academic Year 2025/2026

Acknowledgements

I would like to express my sincere gratitude to all those who contributed, directly or indirectly, to the completion of this work.

I am deeply grateful to my supervisor, Professor [Name], for their availability, valuable guidance, and continuous support throughout this project. Their mentorship allowed me to structure my thinking and deepen my understanding of machine learning applied to finance.

I also extend my thanks to the entire faculty of the Master's Program in Applied Economics and Data Science for the quality of the education provided, and for equipping me with the conceptual and technical tools necessary to conduct this work.

My gratitude goes to the Kaggle community and the researchers of the Machine Learning Group at the Université Libre de Bruxelles (ULB) for making the Credit Card Fraud Detection dataset publicly available.

Finally, I thank my family for their unwavering support and constant encouragement throughout my academic journey.

Abstract

Financial fraud has become one of the major challenges faced by modern banking institutions. With the rapid development of digital banking systems and online payment technologies, fraudulent transactions have increased significantly in recent years.

This project focuses on the detection of fraudulent banking transactions using Machine Learning techniques, particularly the Isolation Forest algorithm. The study uses the Credit Card Fraud Detection dataset obtained from Kaggle, which contains both legitimate and fraudulent transactions.

The main objective of this work is to identify suspicious transactions efficiently while dealing with the highly imbalanced nature of fraud datasets. The implementation and analysis are performed using the R programming language.

The obtained results demonstrate that Isolation Forest is an effective anomaly detection technique capable of detecting fraudulent activities with good performance.

Keywords: Banking Fraud Detection, Machine Learning, Isolation Forest, Anomaly Detection, Credit Card Fraud, R Programming.

Contents

Acknowledgements **1**

Abstract **2**

General Introduction **9**

0.1 Problem Statement . . . 10
0.2 Research Question . . . 10
0.3 Objectives of the Study . . . 10
0.4 Importance of the Study . . . 11

1 Banking Theory and Financial Fraud **12**

1.1 Introduction . . . 12
1.2 Definition and Typologies of Financial Fraud . . . 12
1.2.1 General Definition . . . 12
1.2.2 Credit Card Fraud . . . 12
1.2.3 Other Types of Banking Fraud . . . 13
1.3 Electronic Payment Systems . . . 13
1.3.1 Architecture . . . 13
1.3.2 Vulnerabilities of Digital Payment Systems . . . 13
1.4 Economic Impact of Banking Fraud . . . 14
1.5 Regulatory Framework . . . 14
1.5.1 International Regulations . . . 14
1.5.2 Regulatory Context in Morocco . . . 14
1.6 Traditional Fraud Detection Systems and Their Limits . . . 15
1.7 Chapter Conclusion . . . 15

2 Machine Learning and Anomaly Detection **16**

2.1 Introduction . . . 16
2.2 Taxonomy of Machine Learning Methods . . . 16
2.2.1 Supervised Learning . . . 16
2.2.2 Unsupervised Learning . . . 16
2.2.3 Semi-Supervised Learning . . . 16
2.3 Anomaly Detection: Fundamental Concepts . . . 17
2.3.1 Main Anomaly Detection Methods . . . 17
2.4 The Class Imbalance Problem . . . 17

2.4.1 Definition and Measurement . . . 17
2.4.2 Why Accuracy is Misleading . . . 18
2.4.3 Metrics Adapted to Imbalanced Data . . . 18
2.4.4 Strategies for Handling Imbalance . . . 18
2.5 Literature Review . . . 18
2.5.1 Foundational Work . . . 18
2.5.2 Recent Developments . . . 18
2.6 Methodology . . . 19
2.6.1 General Approach . . . 19
2.6.2 Data Preprocessing and Cleaning . . . 20
2.6.3 Descriptive Statistics for Transactional Data . . . 20
2.6.4 Financial Risk Measurement and VaR . . . 21
2.6.5 Time Series and Volatility Models in Finance . . . 21
2.6.6 Regression and Statistical Inference in Finance . . . 23
2.6.7 Portfolio Theory and Financial Risk . . . 23
2.7 Literature Review . . . 24
2.7.1 Foundational Work . . . 24
2.7.2 Recent Developments . . . 24
2.7.3 Isolation Forest Algorithm . . . 24
2.8 Theoretical Foundation of Isolation Forest . . . 24
2.9 Isolation Through Random Partitioning . . . 25
2.9.1 The Isolation Tree (iTree) . . . 25
2.9.2 Anomaly Score . . . 26
2.9.3 Score Interpretation . . . 26
2.10 Working Principle of the Algorithm . . . 26
2.11 Building the Forest . . . 27
2.11.1 Complete Algorithm . . . 27
2.11.2 Key Parameters . . . 28
2.12 Advantages of Isolation Forest . . . 28
2.12.1 Computational Complexity . . . 28
2.12.2 Robustness to Class Imbalance . . . 29
2.12.3 Resistance to the Curse of Dimensionality . . . 29
2.13 Limitations and Improved Variants . . . 29
2.14 Chapter Conclusion . . . 29

3 Dataset and Methodology in R 30
3.1 Introduction . . . 30
3.2 Dataset Description . . . 30
3.2.1 Source and Context . . . 30
3.2.2 Variable Description . . . 31
3.3 Data Loading and Quality Check in R . . . 31
3.4 Exploratory Data Analysis (EDA) in R . . . 31

3.4.1 Class Distribution . . . 31
3.4.2 Transaction Amount Analysis . . . 32
3.4.3 Temporal Distribution . . . 34
3.4.4 PCA Feature Analysis . . . 34
3.4.5 Correlation Matrix . . . 35
3.5 Data Preprocessing in R . . . 36
3.5.1 Complete Preprocessing Pipeline . . . 36
3.6 Chapter Conclusion . . . 37

4 R Implementation and Results 38
4.1 Introduction . . . 38
4.2 R Packages Used . . . 38
4.3 Model Training . . . 38
4.4 Anomaly Score Computation . . . 39
4.4.1 Anomaly Score Distribution . . . 39
4.5 Model Evaluation . . . 40
4.5.1 Confusion Matrix . . . 40
4.5.2 ROC Curve and AUC . . . 41
4.5.3 Confusion Matrix Visualization . . . 42
4.5.4 Threshold Sensitivity Analysis . . . 43
4.5.5 t-SNE Visualization . . . 44
4.6 Managerial Impacts . . . 46
4.6.1 Implications for Financial Institutions . . . 46
4.6.2 Performance Comparison with the Literature . . . 46
4.7 Chapter Conclusion . . . 46

5 Limitations, Hybrid Extensions, and Perspectives 47
5.1 Introduction . . . 47
5.2 Identified Model Limitations . . . 47
5.2.1 Limited Recall . . . 47
5.2.2 Opacity of Anonymized Features . . . 47
5.2.3 Static Dataset . . . 47
5.2.4 Absence of Behavioral Context . . . 48
5.3 Proposed Hybrid Extensions . . . 48
5.3.1 Isolation Forest + Supervised Random Forest . . . 48
5.3.2 Adding Explainability with SHAP . . . 48
5.3.3 Real-Time Detection (Streaming) . . . 48
5.4 Future Research Perspectives . . . 49
5.5 Chapter Conclusion . . . 49

General Conclusion 50

A Complete R Script 52

B Descriptive Statistics of Variables V1–V28 **57**

List of Figures

3.1 Transaction class distribution on a logarithmic scale 32
3.2 Transaction amount distribution by class (log scale) 33
3.3 Temporal density of normal and fraudulent transactions 34
3.4 Distribution of variables V1–V8 by class 35
3.5 Correlation matrix for variables V1 to V10 36

4.1 Anomaly score distribution for normal and fraudulent transactions 40
4.2 ROC Curve — AUC = 0.9480 42
4.3 Visual confusion matrix — Isolation Forest 43
4.4 Precision, Recall and F1-Score as a function of the decision threshold 44
4.5 t-SNE visualization: separation of fraudulent and normal transactions 45

List of Tables

1.1 Evolution of global card fraud losses (USD billions) 14
1.2 Comparison of fraud detection approaches 15

2.1 Confusion matrix structure 18
2.2 Isolation Forest parameters and their impact 28
2.3 Complexity comparison of anomaly detection algorithms 29

3.1 General characteristics of the dataset 30
3.2 Description of dataset variables 31
3.3 Descriptive statistics of transaction amounts by class 33

4.1 Confusion matrix obtained with the actual contamination threshold 41
4.2 Summary of performance metrics 41
4.3 Impact of the decision threshold on performance metrics 44
4.4 Performance comparison with the literature (Kaggle dataset) 46

B.1 Descriptive statistics for variables V1–V14 57

General Introduction

General Context

The rise of digital financial services has fundamentally transformed the global banking landscape. Online payment platforms, mobile banking applications, and contactless transactions have made financial exchanges faster and more accessible than ever before. However, this rapid digitalization has simultaneously opened new avenues for criminal exploitation, leading to a significant surge in fraudulent activities worldwide [1].

Financial fraud, broadly defined as any illegal act involving deception for monetary gain, represents one of the most pressing threats to modern financial institutions. Credit card fraud alone accounts for billions of dollars in losses annually. According to the Nilson Report, global card fraud losses exceeded $32 billion in 2021 and are projected to surpass $40 billion by 2027 [2]. These figures underscore the urgency of developing robust and automated fraud detection mechanisms.

Traditional rule-based fraud detection systems, while historically adequate, have demonstrated increasing limitations in the face of evolving fraud patterns. Such systems rely on manually defined thresholds and static rules, making them inherently reactive rather than proactive [3]. As fraudulent behavior becomes more sophisticated and adaptive, the need for intelligent, data-driven approaches has become critical.

Machine Learning (ML) has emerged as a powerful alternative for fraud detection, offering the ability to learn complex patterns from large volumes of transactional data. Supervised learning methods such as logistic regression, decision trees, and neural networks have been widely explored in this context [4]. However, these methods typically require labeled data — i.e., transactions explicitly marked as fraudulent — which is often scarce, costly to obtain, and subject to labeling errors.

In contrast, unsupervised anomaly detection techniques offer a compelling advantage: they can identify irregular patterns without relying on pre-labeled fraud examples. Among these, the Isolation Forest algorithm, introduced by Liu et al. (2008), has gained considerable attention due to its computational efficiency and strong performance on high-dimensional, imbalanced datasets [5]. The algorithm operates by isolating observations through random partitioning, under the premise that anomalies — being few and structurally distinct — are easier to isolate than normal instances.

The significance of anomaly detection in financial systems cannot be overstated. As Chandola et al. (2009)[9] aptly noted:"Anomaly detection refers to the problem of finding patterns in data that do not conform to expected behavior. These non-conforming patterns are often referred to

as anomalies, outliers, discordant observations, exceptions, aberrations, surprises, peculiarities, or contaminants in different application domains."

This definition is particularly relevant in the context of banking fraud, where deviations from normal transactional behavior serve as the primary signal for detection.

A persistent challenge in fraud detection is the extreme class imbalance inherent to fraud datasets. In real-world transaction data, fraudulent cases typically represent less than 0.2

This project leverages the publicly available Credit Card Fraud Detection dataset from Kaggle, a benchmark dataset widely used in academic and applied research [7]. The dataset contains transactions made by European cardholders over two days in September 2013, with 492 fraudulent transactions out of 284,807 total — representing a fraud rate of approximately 0.17

The implementation and analysis in this work are carried out entirely using the R programming language, a widely adopted environment for statistical computing and machine learning. R provides a rich ecosystem of libraries suited for anomaly detection tasks, including the isotree and solitude packages, which implement the Isolation Forest algorithm [8].

0.1 Problem Statement

Detecting fraudulent banking transactions is a difficult task because fraud cases are extremely rare compared to legitimate transactions. This creates a highly imbalanced dataset problem.

In fraud detection datasets, the number of fraudulent observations generally represents less than 1% of total transactions. Consequently, traditional classification methods may fail to identify anomalies correctly.

The challenge of this project is therefore to develop a robust anomaly detection model capable of identifying suspicious transactions while minimizing false positives and false negatives.

0.2 Research Question

This study aims to answer the following research question:

How can machine learning techniques help detect fraudulent banking transactions effectively?

0.3 Objectives of the Study

The main objective of this study is to develop an effective approach for detecting fraudulent banking transactions using machine learning techniques, specifically Isolation Forest.

- Identify anomalous transactions that may indicate fraudulent behavior.
- Apply an unsupervised learning method suitable for imbalanced datasets.
- Evaluate the effectiveness of Isolation Forest in fraud detection.
- Contribute to improving the detection of financial fraud in real-world banking systems.

0.4 Importance of the Study

Fraud detection systems play a major role in reducing financial losses and improving transaction security. The use of Machine Learning algorithms allows financial institutions to identify suspicious behaviors more rapidly and accurately.

This project demonstrates how anomaly detection techniques can contribute to modern financial security systems.

Structure of the Report

This report is organized into six parts:

- **Part I** presents the theoretical foundations of the banking sector and financial fraud, including fraud typologies, payment systems, and the regulatory framework.
- **Part II** offers a literature review on machine learning methods applied to anomaly detection, with particular focus on the challenges of class imbalance.
- **Part III** details the mathematical and algorithmic workings of the `IF` algorithm, its theoretical properties and variants.
- **Part IV** describes the Kaggle dataset, the preprocessing methodology in R, and the in-depth exploratory data analysis.
- **Part V** presents the R implementation of the model, the obtained results, their critical analysis, and managuerial implications. Finally,
- **Part VI** addresses the model's limitations, possible hybrid extensions, and future research perspectives.

Chapter 1

Banking Theory and Financial Fraud

1.1 Introduction

The banking sector constitutes the backbone of modern economies. By collecting savings, granting credit, and facilitating payments, banks play a central role in the circulation of wealth. However, the digital transformation of this sector has simultaneously exposed financial institutions to increasingly sophisticated and difficult-to-detect fraud risks.

This chapter establishes the conceptual foundations necessary to understand the phenomenon of banking fraud. We successively examine the definition and typologies of financial fraud, the mechanisms of electronic payment systems, the global economic impact of fraud, and the regulatory framework governing the fight against this phenomenon.

1.2 Definition and Typologies of Financial Fraud

1.2.1 General Definition

Financial fraud is defined as any intentional act aimed at obtaining an illicit financial advantage at the expense of a third party, through deception, forgery, or abuse of trust. In the banking context, it encompasses a broad spectrum of criminal activities.

Definition 1 (Banking Fraud)**.** *Banking fraud refers to any deliberate action, whether by act or omission, aimed at improperly obtaining funds, services, or assets held by a financial institution or its customers, through deception, misrepresentation, or manipulation of information systems.*

1.2.2 Credit Card Fraud

Credit card fraud represents one of the most prevalent forms of financial fraud in the digital era:

- **Card Not Present (CNP) fraud**: use of card data without physical possession of the card, particularly in online purchases. This type now represents the majority of fraud cases.
- **Card Present fraud**: use of a counterfeit physical card, typically obtained through magnetic cloning (*skimming*).

- **Account Takeover (ATO)**: fraudulent takeover of an existing bank account through credential theft or phishing.
- **Friendly fraud**: fraudulent dispute of a legitimate transaction by the cardholder (chargeback fraud).
- **New Account Fraud**: opening an account under a stolen or fictitious identity with no intention of repaying granted credit.

1.2.3 Other Types of Banking Fraud

- **Documentary fraud**: forgery of cheques, banknotes, or identity documents.
- **Internal fraud**: misconduct committed by employees of financial institutions.
- **Money laundering**: concealment of the illicit origin of funds through apparently legitimate transactions.
- **Cyber fraud**: computer attacks targeting banking systems (malware, ransomware, Man-in-the-Middle attacks).

1.3 Electronic Payment Systems

1.3.1 Architecture

An electronic payment system involves several interconnected actors:

- **Cardholder**: the consumer initiating the transaction.
- **Merchant**: the entity accepting the payment.
- **Acquiring bank**: the merchant's financial institution.
- **Issuing bank**: the institution that issued the cardholder's card.
- **Payment network**: Visa, Mastercard, American Express, etc.

A card transaction follows a precise flow: authorization, authentication, clearing, and settlement. Each step generates data that can be exploited for fraud detection.

1.3.2 Vulnerabilities of Digital Payment Systems

- Absence of physical verification in online transactions (CNP).
- Weaknesses in authentication protocols (credential stuffing, brute force attacks).
- Compromise of merchant databases (data breaches).
- Exploitation of the delay between transaction and detection.

1.4 Economic Impact of Banking Fraud

According to the Nilson Report (2022), global card fraud losses reached **$32.34 billion** in 2021, with projections exceeding **$40 billion** by 2027.

Table 1.1: Evolution of global card fraud losses (USD billions)

Year	Losses (USD Bn)	Growth (%)
2016	22.80	—
2017	23.97	5.1%
2018	24.26	1.2%
2019	28.65	18.1%
2020	28.58	-0.2%
2021	32.34	13.2%
2027*	40.62*	(projection)

*Projection. Source: The Nilson Report, 2022.

Beyond direct losses (lexisnexis2022), fraud generates significant indirect costs: operational costs (chargebacks, investigations), reputational damage, compliance costs, and false positive costs from wrongfully blocked legitimate transactions. It is estimated that for every dollar of fraud, institutions spend between **$3 and $4** in remediation.

1.5 Regulatory Framework

1.5.1 International Regulations

- **PCI-DSS**: international standard defining security requirements for all entities processing payment card data.
- **GDPR** (2018): European regulation on personal data protection. In the fraud context, it mandates minimization and pseudonymization of transactional data — which explains the PCA anonymization in the Kaggle dataset.
- **PSD2**: European directive mandating Strong Customer Authentication (SCA) for electronic payments within the SEPA area.
- **Basel III / Basel IV**: international banking accords governing operational risk management, including fraud risks.

1.5.2 Regulatory Context in Morocco

- **Bank Al-Maghrib**: central bank of the Kingdom, responsible for regulating and supervising the banking system, issuing circulars on information security and operational risk.
- **Law No. 43-20** relating to trust services for electronic transactions.

- **CMI** (Centre Monétique Interbancaire): organization managing interbank electronic transactions in Morocco, operating real-time fraud detection systems.

1.6 Traditional Fraud Detection Systems and Their Limits

The first fraud detection systems relied on manually defined business rules. While transparent and interpretable, they are **static**, **reactive**, and incapable of adapting to new fraudulent strategies (bolton2002).

Table 1.2: Comparison of fraud detection approaches

Approach	**Labeled Data**	**Scalability**	**Adaptability**	**False Positives**
Rule-based	No	Low	Very low	High
Logistic Regression	Yes	Good	Low	Moderate
Random Forest	Yes	Good	Moderate	Low
SVM	Yes	Low	Low	Variable
Isolation Forest	**No**	**Excellent**	**Good**	**Controllable**

1.7 Chapter Conclusion

This chapter has established the economic and regulatory context of banking fraud. The growing complexity of fraudulent behaviors, combined with the extreme imbalance of transactional data, clearly requires innovative AI-based approaches. The following chapter presents the theoretical foundations of machine learning applied to anomaly detection.

Chapter 2

Machine Learning and Anomaly Detection

2.1 Introduction

Machine learning (ML) refers to a set of methods enabling a computer system to learn from data without being explicitly programmed for each task. Since the 2010s, ML has established itself as the reference approach for fraud detection. This chapter presents the theoretical foundations of ML applied to anomaly detection, with particular emphasis on the challenges posed by class imbalance in financial data.

2.2 Taxonomy of Machine Learning Methods

2.2.1 Supervised Learning

Supervised learning trains a model on a labeled dataset where each observation is associated with a known class. The most commonly used supervised algorithms in fraud detection include logistic regression, decision trees, Random Forest, Gradient Boosting (XGBoost, LightGBM), and neural networks.

Main limitation: these methods require large amounts of labeled data, which is difficult to obtain in the banking fraud context.

2.2.2 Unsupervised Learning

Unsupervised learning requires no labels. In the fraud detection context, it is particularly relevant because labeled fraud data is rare, fraud patterns evolve constantly, and detection can operate in real time without frequent retraining.

2.2.3 Semi-Supervised Learning

An intermediate approach combining abundant unlabeled data with a limited number of labeled examples, leveraging the global structure of the data while benefiting from partial available information.

2.3 Anomaly Detection: Fundamental Concepts

Definition 2 (Anomaly)**.** *An anomaly (or outlier) is an observation that deviates significantly from the expected behavior in a dataset. In a financial context, a transactional anomaly is behavior that differs substantially from the habitual patterns of an account or a population of accounts.*

Chandola et al. (2009) identify three types of anomalies:

- **Point anomalies**: individual atypical observations — the most common case in fraud detection.
- **Contextual anomalies**: observations that are normal in one context but abnormal in another (e.g., a $10,000 withdrawal is normal for a business, suspicious for an individual).
- **Collective anomalies**: sequences of observations that, taken individually, appear normal, but collectively constitute an anomaly.

2.3.1 Main Anomaly Detection Methods

Distance-Based — LOF

The LOF (*Local Outlier Factor*) **?** measures the degree of isolation of a point relative to its k nearest neighbors:

$$\mathrm{LOF}_k(p) = \frac{\displaystyle\sum_{o \in N_k(p)} \frac{\mathrm{lrd}_k(o)}{\mathrm{lrd}_k(p)}}{|N_k(p)|} \tag{2.1}$$

Limitation: high computational cost $O(n^2)$, unsuitable for large transactional volumes.

Isolation-Based — Isolation Forest

IF, the central method of this study, rests on a fundamentally different principle: isolating observations rather than modeling normal data distribution. Fully detailed in Part III.

2.4 The Class Imbalance Problem

2.4.1 Definition and Measurement

In the Kaggle dataset used in this study:

$$\text{Imbalance Ratio} = \frac{N_{\text{normal}}}{N_{\text{fraud}}} = \frac{284{,}315}{492} \approx 578 : 1 \tag{2.2}$$

2.4.2 Why Accuracy is Misleading

A naive classifier always predicting "Normal" achieves:

$$\text{Accuracy}_{\text{naive}} = \frac{284{,}315}{284{,}807} \approx 99.83\% \tag{2.3}$$

yet detects zero fraud. Specialized metrics are therefore mandatory.

2.4.3 Metrics Adapted to Imbalanced Data

Table 2.1: Confusion matrix structure

		Actual Class	
		Negative (Normal)	**Positive (Fraud)**
Predicted Class	**Negative**	True Negative (TN)	False Negative (FN)
	Positive	False Positive (FP)	True Positive (TP)

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN} \qquad \text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{2.4}$$

In the fraud context, **Recall** is particularly critical: a missed fraud (FN) has severe financial consequences. The **AUC-ROC** summarizes the overall discriminative power of the model, with AUC > 0.9 considered excellent for fraud detection.

2.4.4 Strategies for Handling Imbalance

- **Oversampling (SMOTE) ?**: synthetic generation of minority class examples.
- **Undersampling**: reduction of the majority class.
- **Class weighting**: stronger penalization of minority class errors.
- **Anomaly detection methods**: treating the problem as outlier detection — the approach adopted in this study, which entirely sidesteps the imbalance issue.

2.5 Literature Review

2.5.1 Foundational Work

The first ML applications to fraud detection date back to the 1990s. **? ?** demonstrated the superiority of neural networks over rule-based systems. **? ?** proposed CARDWATCH, a neural network-based system for credit card fraud detection.

2.5.2 Recent Developments

- **Ensemble methods**: Random Forest and XGBoost have shown superior benchmark performance **?**.

- **Deep learning**: LSTM networks and variational autoencoders have been proposed to capture temporal patterns in transactional sequences **?**.

2.6 Methodology

This section presents the methodology adopted to build the fraud detection model. The approach is based on an unsupervised machine learning technique, specifically the Isolation Forest algorithm.

2.6.1 General Approach

IF Is an anomaly detection algorithm introduced by Liu, Ting, and Zhou in 2008 at the IEEE International Conference on Data Mining (ICDM) **?**. Rather than modeling the distribution of normal data to infer anomalies by deviation, IF directly seeks to *isolate* atypical observations, exploiting two key properties of anomalies:

- They are **few** — a minority characteristic.
- They are **different** — structurally distinct from normal data.

The overall process of the study follows these main steps:

- Data collection from a Kaggle credit card fraud dataset
- Data preprocessing and cleaning
- Exploratory Data Analysis (EDA)
- Application of Isolation Forest model
- Detection of anomalies (fraudulent transactions)
- Evaluation of model performance
- **Observational, not experimental**: financial data are the outcome of market processes, not designed experiments. This limits causal inference.
- **Noise and volatility**: prices and transaction amounts fluctuate due to market microstructure effects, rendering the signal-to-noise ratio inherently low.
- **Non-stationarity**: price levels are generally non-stationary. Working with *returns* rather than price levels improves the statistical properties of the data.
- **Dependence over time**: financial observations are ordered sequences exhibiting autocorrelation and volatility clustering.

In the context of banking transaction data — such as the Kaggle dataset used in this study — transactions constitute a specific type of financial data: *Banking and FinTech data*, characterized by their granularity and their usefulness for micro-level analysis such as credit risk assessment and fraud detection.

2.6.2 Data Preprocessing and Cleaning

Data preprocessing refers to the set of operations applied to raw data to improve their quality and suitability for analysis. In finance, ignoring preprocessing can lead to biased estimates and unstable models. The main preprocessing steps are:

- **Missing value treatment**: missing values occur when observations are unavailable or not recorded. Common strategies include deletion of incomplete observations, and mean imputation:

$$x_i^* = \begin{cases} x_i & \text{if observed} \\ \bar{x} & \text{if missing} \end{cases} \tag{2.5}$$

  In the Kaggle Credit Card dataset, no missing values were detected, which confirms the quality of the data source.

- **Duplicate removal**: duplicate rows may introduce artificial weighting of certain observations. In our dataset, 1,081 duplicate rows were identified and removed.

- **Scaling and standardization (z-score normalization)**: financial variables are often measured on very different scales. Standardization ensures that no variable dominates others purely due to its magnitude:

$$z_i = \frac{x_i - \mu}{\sigma} \tag{2.6}$$

  This is particularly important for distance-based and optimization models. In this study, the variables `Amount` and `Time` were standardized using this formula prior to modeling.

- **Data transformation**: many financial time series exhibit trends or changing variance. Logarithmic transformations ($y = \ln(x)$) and first differences ($\Delta x_t = x_t - x_{t-1}$) are applied to stabilize statistical properties and improve stationarity.

2.6.3 Descriptive Statistics for Transactional Data

Descriptive statistics summarize the structure and quality of a dataset. In the context of fraud detection, they allow comparing the distributional properties of fraudulent versus normal transactions across four families of statistics:

- **Central tendency**: mean $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$ and median. In our dataset, fraudulent transactions have a lower median amount (€9.25 vs €22.00), revealing a concentration at small amounts.

- **Dispersion**: variance $s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$ and standard deviation $s = \sqrt{s^2}$. Both classes exhibit similar standard deviations (≈€250), but their distributions differ substantially in shape.

- **Shape**: skewness and kurtosis. Fraudulent transaction amounts present a right-skewed, leptokurtic distribution — consistent with the stylized facts of financial data.

- **Dependence**: covariance and correlation. The PCA-derived features V1–V28 are by construction orthogonal (uncorrelated), which simplifies the modeling task.

2.6.4 Financial Risk Measurement and VaR

The notion of risk measurement is central to the course framework of Data Science and Finance. Two key risk measures are particularly relevant in the fraud detection context:

Value-at-Risk (VaR)

The Value-at-Risk at confidence level α is defined as:

$$\text{VaR}_\alpha = \inf\{l : \mathbb{P}(L \leq l) \geq \alpha\} \tag{2.7}$$

In the context of banking transaction anomaly detection, the decision threshold of the IF model can be interpreted analogously to a VaR: it defines the quantile of the anomaly score distribution beyond which a transaction is classified as fraudulent. Specifically, we set:

$$\text{threshold} = Q_{1-\hat{p}}(\text{anomaly scores}) \tag{2.8}$$

where $\hat{p} = 0.001728$ is the estimated contamination rate (analogous to the loss probability in VaR).

Credit Risk and Expected Loss

Credit risk is often modeled using the expected loss framework:

$$EL = PD \times LGD \times EAD \tag{2.9}$$

where PD is the Probability of Default, LGD is the Loss Given Default, and EAD is the Exposure at Default. In fraud detection, the anomaly score produced by IF can be interpreted as a proxy for the Probability of Fraud (PF), which combined with the transaction amount (EAD) gives a risk-weighted fraud alert score.

2.6.5 Time Series and Volatility Models in Finance

ARIMA Models

The ARIMA(p, d, q) framework models the differenced series $Y_t = \Delta^d X_t$ with an ARMA(p, q) model:

$$\Phi(B)\Delta^d X_t = \Theta(B)\varepsilon_t \tag{2.10}$$

where $\Phi(B) = 1 - \phi_1 B - \cdots - \phi_p B^p$ and $\Theta(B) = 1 + \theta_1 B + \cdots + \theta_q B^q$. In the transactional fraud context, the temporal feature `Time` reflects an ordering of transactions that could be exploited through ARIMA-type modeling to detect behavioral drift over time.

Model selection uses the Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC):

$$\text{AIC} = 2k - 2\ln(\hat{L}), \qquad \text{BIC} = k\ln(n) - 2\ln(\hat{L}) \tag{2.11}$$

where k is the number of parameters and $\hat{L}$ is the maximized likelihood.

GARCH Models and Volatility Clustering

Standard time series models assume *homoskedasticity*: $\text{Var}(\varepsilon_t|\mathcal{F}_{t-1}) = \sigma^2$. In finance, however, the error variance is **not** constant. The ARCH(q) model (Engle, 1982) captures this through:

$$r_t = \mu + \varepsilon_t \tag{2.12}$$

$$h_t = \alpha_0 + \alpha_1\varepsilon_{t-1}^2 + \cdots + \alpha_q\varepsilon_{t-q}^2 \tag{2.13}$$

The GARCH$(1,1)$ model (Bollerslev, 1986) extends this by adding memory of past variance:

$$h_t = \omega + \alpha\varepsilon_{t-1}^2 + \beta h_{t-1} \tag{2.14}$$

For stability, the stationarity condition requires $\alpha + \beta < 1$, which ensures the long-run average variance $\sigma^2 = \frac{\omega}{1-\alpha-\beta}$ exists. The persistence $(\alpha + \beta)$ measures how slowly volatility shocks die out.

Relevance to fraud detection: volatility clustering — the tendency for high-volatility periods to cluster together — is a stylized fact of financial returns. In a fraud detection context, *fraud clustering* (periods with elevated fraud rates) exhibits an analogous pattern. Anomaly scores produced by IF can be analyzed over time to detect such clustering effects.

GARCH and VaR: GARCH is critical for risk management. The 1-day Value-at-Risk at 95% confidence is:

$$\text{VaR}_{t+1} = \mu + \Phi^{-1}(0.05) \cdot \sqrt{h_{t+1}} \tag{2.15}$$

where $\sqrt{h_{t+1}}$ is the GARCH forecast of conditional volatility. This allows for Dynamic Risk Limits — a concept directly applicable to adaptive fraud threshold calibration.

Extensions: EGARCH and GJR-GARCH

The leverage effect — the empirical observation that negative returns generate more volatility than positive returns of the same magnitude — cannot be captured by standard GARCH. Two extensions address this:

- **GJR-GARCH**: adds a dummy variable for negative shocks.
- **EGARCH**: models $\ln(h_t)$ to ensure positivity and capture asymmetry.

2.6.6 Regression and Statistical Inference in Finance

Linear Regression

Regression models how a target variable depends on one or more predictors. The simple linear regression is:

$$Y_t = \beta_0 + \beta_1 X_t + \varepsilon_t \tag{2.16}$$

Estimated by Ordinary Least Squares (OLS), which minimizes:

$$\min_{\beta_0,\beta_1} \sum_{t=1}^{T} (Y_t - \beta_0 - \beta_1 X_t)^2 \tag{2.17}$$

The OLS estimators are:

$$\hat{\beta}_1 = \frac{\sum_{t=1}^{T}(X_t - \bar{X})(Y_t - \bar{Y})}{\sum_{t=1}^{T}(X_t - \bar{X})^2}, \qquad \hat{\beta}_0 = \bar{Y} - \hat{\beta}_1 \bar{X} \tag{2.18}$$

Statistical Inference

Testing whether a factor matters uses:

$$H_0 : \beta_j = 0 \qquad H_1 : \beta_j \neq 0 \tag{2.19}$$

The t-statistic is $t = \frac{\hat{\beta}_j}{\mathrm{SE}(\hat{\beta}_j)}$. Model fit is measured by:

$$R^2 = 1 - \frac{SSE}{SST} = 1 - \frac{\sum(Y_t - \hat{Y}_t)^2}{\sum(Y_t - \bar{Y})^2} \tag{2.20}$$

In the fraud detection context, a logistic regression baseline can be used to compare with the unsupervised IF approach, providing a benchmark in terms of AUC-ROC and classification metrics.

2.6.7 Portfolio Theory and Financial Risk

Portfolio Return and Variance

For a two-asset portfolio with weights w and $(1 - w)$:

$$R_p = wR_1 + (1 - w)R_2 \tag{2.21}$$

$$\sigma_p^2 = w^2\sigma_1^2 + (1 - w)^2\sigma_2^2 + 2w(1 - w)\rho\sigma_1\sigma_2 \tag{2.22}$$

The term $(\alpha + \beta)$ in GARCH is often called *persistence*. A high persistence means volatility shocks die out slowly — a relevant concept for detecting sustained fraud campaigns.

Markowitz Optimization

The classical Markowitz portfolio optimization problem is:

$$\min_w w^\top \Sigma w \quad \text{s.t.} \quad \mathbb{E}(R_p) = \mu^*, \quad \sum w_i = 1 \tag{2.23}$$

This quadratic optimization problem yields the efficient frontier — the set of portfolios achieving maximum expected return for a given risk level. The decision-threshold optimization in fraud detection (maximizing F1-Score across thresholds) is structurally analogous: it seeks the optimal operating point on the precision-recall tradeoff curve, just as Markowitz seeks the optimal point on the risk-return frontier.

2.7 Literature Review

2.7.1 Foundational Work

The first ML applications to fraud detection date back to the 1990s. **?** **?** demonstrated the superiority of neural networks over rule-based systems. **?** **?** proposed CARDWATCH, a neural network-based system for credit card fraud detection.

2.7.2 Recent Developments

- **Ensemble methods**: Random Forest and XGBoost have shown superior benchmark performance **?**.
- **Deep learning**: LSTM networks and variational autoencoders have been proposed to capture temporal patterns in transactional sequences **?**.

2.7.3 Isolation Forest Algorithm

Isolation Forest is an unsupervised anomaly detection algorithm designed to identify outliers in a dataset. Unlike traditional methods, it does not rely on labeled data during training.

The algorithm works by randomly selecting features and splitting values to isolate observations. Anomalies are easier to isolate because they are rare and different from normal patterns.

The anomaly score is computed based on the number of splits required to isolate a data point. A lower number of splits indicates a higher probability of fraud.

2.8 Theoretical Foundation of Isolation Forest

Isolation Forest is an unsupervised anomaly detection algorithm introduced by Liu, Ting, and Zhou in 2008 **?**. The algorithm is specifically designed to isolate anomalous observations instead of profiling normal data points.

The fundamental idea behind Isolation Forest is based on the assumption that anomalies are both rare and significantly different from normal observations. Consequently, anomalous data points are easier to isolate through random partitioning.

The algorithm constructs multiple random binary trees, known as isolation trees (iTrees). For each tree, a random feature is selected, followed by a random split value between the minimum and maximum values of the selected feature.

The partitioning process continues recursively until each observation becomes isolated. Since anomalies differ considerably from regular observations, they generally require fewer partitions to become isolated. Therefore, anomalous observations tend to have shorter path lengths within the trees.

In contrast, normal observations usually require a greater number of partitions because they belong to dense regions of the dataset. The average path length across all trees is then used to compute an anomaly score for each observation.

The anomaly score is defined as follows:

$$s(x, n) = 2^{-c(n)E(h(x))}$$

- $s(x, n)$ represents the anomaly score,
- $E(h(x))$ is the average path length of observation x,
- $c(n)$ is the average path length of unsuccessful searches in Binary Search Trees,
- n represents the number of observations.

An anomaly score close to 1 indicates a high probability that the observation is anomalous, whereas values significantly lower than 0.5 generally correspond to normal observations.

The simplicity and computational efficiency of Isolation Forest make it particularly suitable for large-scale financial fraud detection systems.

2.9 Isolation Through Random Partitioning

2.9.1 The Isolation Tree (iTree)

An Isolation Tree is constructed recursively:

1. Randomly select a feature q from the p available features.
2. Randomly select a split point $\xi \in [\min(q), \max(q)]$.
3. Partition observations into two subsets: $\{x : x_q < \xi\}$ and $\{x : x_q \geq \xi\}$.
4. Repeat recursively until each observation is isolated (leaf node) or a maximum depth $l_{\max}$ is reached.

2.9.2 Anomaly Score

Definition 3 (Path Length)**.** *The path length $h(x)$ of an observation x in an iTree is the number of edges traversed from the root to the leaf node isolating x.*

The expected path length for a Binary Search Tree (BST) of n elements is:

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n} \tag{2.24}$$

where $H(i) = \ln(i) + \gamma$ is the harmonic number ($\gamma \approx 0.5772$: Euler-Mascheroni constant).
The **anomaly score** for observation x is:

$$\boxed{s(x,n) = 2^{-\frac{E[h(x)]}{c(n)}}} \tag{2.25}$$

where $E[h(x)]$ is the average path length across all iTrees in the forest.

2.9.3 Score Interpretation

- $s \to 1$: $E[h(x)] \ll c(n)$ $\Rightarrow$ observation isolated quickly $\Rightarrow$ **likely anomaly**
- $s \to 0.5$: $E[h(x)] \approx c(n)$ $\Rightarrow$ average path length $\Rightarrow$ **normal observation**
- $s \to 0$: $E[h(x)] \gg c(n)$ $\Rightarrow$ difficult to isolate $\Rightarrow$ **very normal**

2.10 Working Principle of the Algorithm

The Isolation Forest algorithm operates by constructing an ensemble of random isolation trees. Each tree is built by recursively partitioning the dataset using randomly selected features and random split values.

The process begins by randomly selecting a variable from the dataset. A split value is then randomly chosen within the range of the selected variable. This operation divides the data into two separate partitions.

The recursive partitioning process continues until one of the following conditions is reached:

- The observation becomes isolated.
- The maximum tree height is reached.
- The partition contains only one observation.

Anomalous observations generally require fewer partitions to become isolated because they are rare and located far from dense clusters of normal data. Consequently, they tend to produce shorter path lengths within the isolation trees.

On the other hand, normal observations belong to dense regions of the dataset and therefore require more partitions before isolation occurs.

The final anomaly score of an observation is computed by averaging the path lengths obtained across all isolation trees in the forest.

The complete workflow of the Isolation Forest algorithm can be summarized as follows:

1. Randomly select a subset of observations from the dataset.
2. Construct multiple isolation trees using random partitioning.
3. Compute the path length for each observation.
4. Calculate the anomaly score based on average path lengths.
5. Classify observations as normal or anomalous.

The algorithm is particularly efficient because it does not rely on distance calculations or density estimation, which are often computationally expensive in large datasets.

2.11 Building the Forest

2.11.1 Complete Algorithm

IF is an ensemble of t iTrees built independently on sub-samples of size ψ drawn without replacement.

```
# Build the isolation forest
IsolationForest(X, t, psi):
  Forest <- {}
  for i = 1 to t do:
    X_sample <- random_sample(X, size = psi)
    tree     <- iTree(X_sample, depth = 0, l_max)
    Forest   <- Forest U {tree}
  return Forest

# Build one isolation tree
iTree(X, depth, l_max):
  if depth >= l_max or |X| <= 1:
    return LeafNode(size = |X|)
  q  <- random_feature()
  xi <- random_value(min(X_q), max(X_q))
  X_left  <- {x in X : x_q < xi}
  X_right <- {x in X : x_q >= xi}
  return InternalNode(
    left    = iTree(X_left,  depth+1, l_max),
    right   = iTree(X_right, depth+1, l_max),
    feature = q,  split = xi)
```

Listing 2.1: Pseudo-code of the Isolation Forest algorithm

2.11.2 Key Parameters

Table 2.2: Isolation Forest parameters and their impact

Parameter	Typical Value	Description
`ntrees` (t)	100	Number of iTrees. Marginal gain is negligible beyond 100.
`sample_size` (ψ)	256	Sample size per tree. Value of 256 is optimal per Liu et al.
`contamination`	Actual proportion	Expected anomaly proportion; determines the decision threshold

2.12 Advantages of Isolation Forest

Isolation Forest presents several advantages that make it highly suitable for fraud detection applications in the banking sector.

One of the main advantages of the algorithm is its computational efficiency. Unlike many traditional anomaly detection methods, Isolation Forest has low time complexity and can process large-scale datasets rapidly. This characteristic is particularly important in banking systems where millions of transactions are generated daily.

Another important advantage is its effectiveness on highly imbalanced datasets. In fraud detection problems, fraudulent transactions represent only a very small proportion of total observations. Traditional classification algorithms often struggle with such imbalance, whereas Isolation Forest directly focuses on identifying rare and abnormal observations.

Furthermore, the algorithm performs well in high-dimensional datasets containing numerous variables. Since banking transaction data often involve multiple features, this capability is essential for accurate fraud detection.

Isolation Forest also does not rely on distance measures or density estimation. As a result, it is less sensitive to the curse of dimensionality compared to clustering-based anomaly detection methods.

In addition, the algorithm is relatively robust to noise and irrelevant features. This robustness improves its practical applicability in real-world financial environments where transaction data may contain noisy information.

According to Liu et al. **?**, Isolation Forest achieves strong anomaly detection performance while maintaining low computational costs, making it an efficient solution for large-scale anomaly detection tasks.

The simplicity of implementation and the availability of specialized R packages such as `isotree` also facilitate the practical application of the algorithm in this project.

2.12.1 Computational Complexity

IF has a training complexity of $O(t \cdot \psi \cdot \log \psi)$, quasi-linear in n for $\psi = 256$.

Table 2.3: Complexity comparison of anomaly detection algorithms

Algorithm	**Training**	**Prediction**
LOF	$O(n^2)$	$O(n)$
One-Class SVM	$O(n^2)$ to $O(n^3)$	$O(n_{sv})$
DBSCAN	$O(n \log n)$	$O(n \log n)$
Isolation Forest	$O(t \cdot \psi \log \psi)$	$O(t \cdot \log \psi)$

2.12.2 Robustness to Class Imbalance

Unlike supervised methods, IF does not rely on the relative frequency of classes. Its performance is therefore not degraded by the extreme imbalance characteristic of fraud datasets.

2.12.3 Resistance to the Curse of Dimensionality

Liu et al. (2008) demonstrated that IF maintains good performance up to several hundred dimensions, whereas LOF and One-Class SVM deteriorate rapidly at high dimensionality **?**.

2.13 Limitations and Improved Variants

- **Masking effect**: difficulties with dense clusters containing embedded anomalies.
- **Uninformative features**: random feature selection may be suboptimal in very high-dimensional data.
- **Contamination threshold**: the actual fraud rate is rarely known in advance.

Improved variants include **Extended Isolation Forest ?** (oblique cuts reducing geometric artifacts), **SCiForest** (cuts in random subspaces), and **RRCF** (Robust Random Cut Forest, adapted for real-time streaming).

2.14 Chapter Conclusion

This chapter has presented the mathematical and algorithmic foundations of IF. Its combination of conceptual simplicity, computational efficiency, and robustness to imbalanced data makes it particularly suited for banking fraud detection.

Chapter 3

Dataset and Methodology in R

3.1 Introduction

This chapter describes the Kaggle Credit Card Fraud Detection dataset, presents the Exploratory Data Analysis (EDA) conducted entirely in R, and details the preprocessing pipeline applied prior to modeling.

3.2 Dataset Description

3.2.1 Source and Context

The dataset is available at https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud, assembled by the Machine Learning Group of the Université Libre de Bruxelles (ULB) under the DEFEATFRAUD project. It contains credit card transactions made by European cardholders during **September 2013** over two consecutive days.

Table 3.1: General characteristics of the dataset

Characteristic	**Value**
Total transactions	284,807
Features	31
Normal transactions	284,315 (99.83%)
Fraudulent transactions	492 (0.17%)
Imbalance ratio	$\approx 578 : 1$
Missing values	0
Duplicates removed	1,081
Period	September 2013 (2 days)

3.2.2 Variable Description

Table 3.2: Description of dataset variables

Variable	Type	Description
`Time`	Numeric	Seconds elapsed since the 1st transaction
`V1–V28`	Numeric	PCA components (anonymized)
`Amount`	Numeric	Transaction amount (in euros)
`Class`	Binary	0 = Normal, 1 = Fraud

Features V1 to V28 are the result of a PCA transformation applied to the original data to protect cardholder privacy. The only non-transformed features are `Time` and `Amount`.

3.3 Data Loading and Quality Check in R

```r
library(tidyverse)
library(scales)

# Load the dataset
data <- read_csv("creditcard.csv")
cat("Rows    :", nrow(data), "\n")   # 284807
cat("Columns :", ncol(data), "\n")  # 31

# Check missing values
missing_values <- colSums(is.na(data))
cat("Missing values :", sum(missing_values), "\n")  # 0

# Check duplicates
cat("Duplicates :", sum(duplicated(data)), "\n")
data <- data %>% distinct()

# Explore structure
glimpse(data)
```

Listing 3.1: Data loading and quality verification in R

The dataset contains **no missing values**. However, **1,081 duplicate rows** were detected and removed, leaving 283,726 unique transactions for analysis, confirming its quality for analysis.

3.4 Exploratory Data Analysis (EDA) in R

3.4.1 Class Distribution

```r
# Encode the target variable
data$Class <- factor(data$Class, levels = c(0, 1),
                     labels = c("Normal", "Fraud"))

table_class <- table(data$Class)
```

```r
prop_class  <- prop.table(table_class) * 100
print(table_class)   # Normal: 284315 | Fraud: 492
print(round(prop_class, 4))  # 99.8272% | 0.1728%

# Visualization -- class distribution (log scale)
p1 <- ggplot(data, aes(x = Class, fill = Class)) +
  geom_bar() +
  scale_y_log10(labels = label_number()) +
  scale_fill_manual(values = c("Normal" = "#2ecc71",
                               "Fraud"  = "#e74c3c")) +
  theme_minimal(base_size = 12) +
  labs(title    = "Transaction Class Distribution (Log Scale)",
       subtitle = paste0("Normal: ", table_class[1],
                         "  |  Fraud: ", table_class[2]),
       x = "Class", y = "Number of Transactions (Log)")
print(p1)
ggsave("fig01_class_distribution.png", p1,
       width = 8, height = 5, dpi = 300)
```

Listing 3.2: Class distribution analysis and visualization



Figure 3.1: Transaction class distribution on a logarithmic scale

The dataset reveals an extreme imbalance: **284,315 normal transactions** (99.83%) versus **492 fraudulent transactions** (0.17%).

3.4.2 Transaction Amount Analysis

```r
# Descriptive statistics by class
stats_by_class <- data %>%
  group_by(Class) %>%
  summarise(
    n         = n(),
    mean_amt  = round(mean(Amount), 2),
```

```r
    median_amt = round(median(Amount), 2),
    sd_amt     = round(sd(Amount), 2),
    max_amt    = round(max(Amount), 2)
  )
print(stats_by_class)

# Violin + boxplot
p3 <- ggplot(data, aes(x = Class, y = Amount, fill = Class)) +
  geom_violin(alpha = 0.7) +
  geom_boxplot(width = 0.1, fill = "white", alpha = 0.8) +
  scale_y_log10(labels = label_dollar()) +
  scale_fill_manual(values = c("Normal" = "#3498db",
                               "Fraud"  = "#e74c3c")) +
  theme_minimal(base_size = 12) +
  labs(title = "Transaction Amount Distribution by Class",
       x = "Class", y = "Amount (Log Scale)")
print(p3)
ggsave("fig03_amount_by_class.png", p3,
       width = 8, height = 5, dpi = 300)
```

Listing 3.3: Amount analysis by class

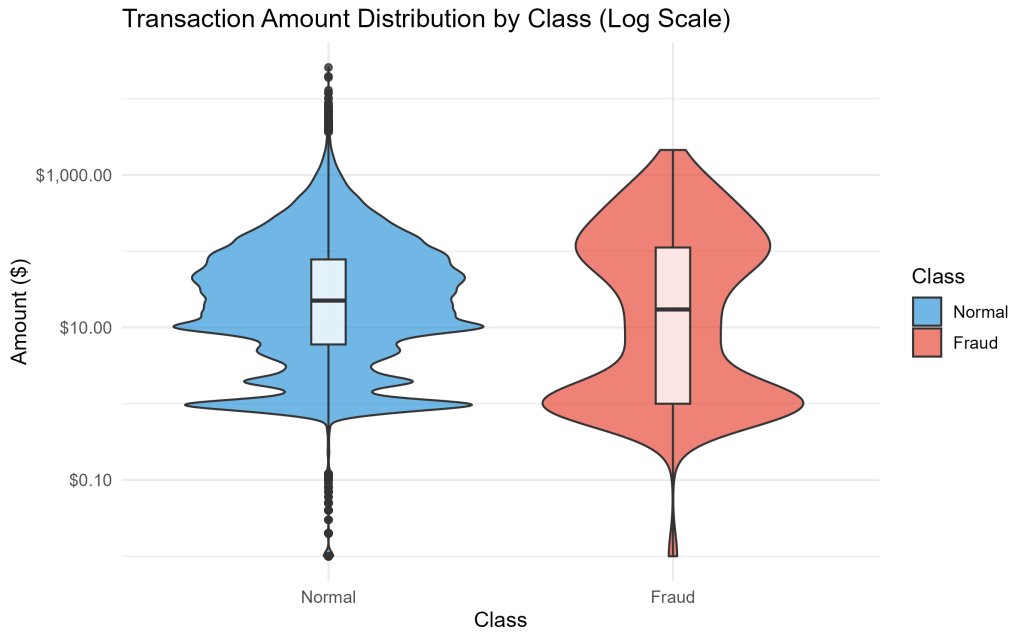


Figure 3.2: Transaction amount distribution by class (log scale)

Table 3.3: Descriptive statistics of transaction amounts by class

Class	Mean (€)	Median (€)	Std Dev	Max (€)
Normal	88.29	22.00	250.12	25,691.16
Fraud	122.21	9.25	256.68	2,125.87

Fraudulent transactions have a **lower median** (€9.25 vs €22.00), suggesting a concentration on small amounts, yet a **higher mean**, indicating a right-skewed distribution with a few high-value fraud cases.

3.4.3 Temporal Distribution

```
p4 <- ggplot(data, aes(x = Time / 3600, fill = Class)) +
  geom_density(alpha = 0.6) +
  scale_fill_manual(values = c("Normal" = "#3498db",
                               "Fraud"  = "#e74c3c")) +
  theme_minimal(base_size = 12) +
  labs(title = "Temporal Distribution of Transactions",
       x = "Time (hours)", y = "Density")
print(p4)
ggsave("fig04_temporal_distribution.png", p4,
       width = 9, height = 4, dpi = 300)
```

Listing 3.4: Temporal distribution of transactions

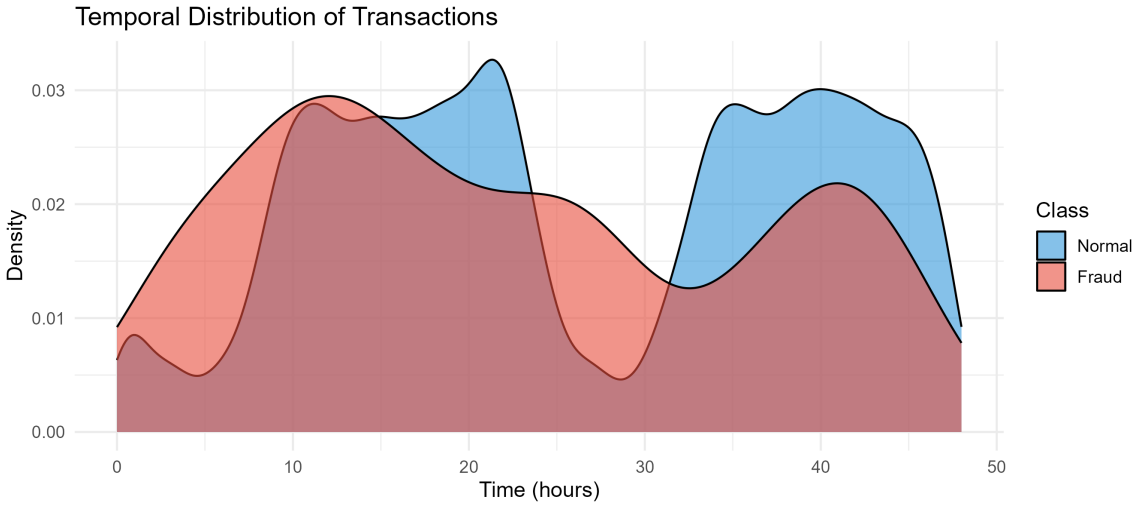


Figure 3.3: Temporal density of normal and fraudulent transactions

Normal transactions exhibit a bimodal pattern corresponding to the two observed days. Fraudulent transactions show a more uniform distribution, suggesting fraud is perpetrated at all hours.

3.4.4 PCA Feature Analysis

```
library(reshape2)

data_long <- data %>%
  select(Class, V1, V2, V3, V4, V5, V6, V7, V8) %>%
  pivot_longer(cols = -Class,
               names_to  = "Variable",
               values_to = "Value")

p5 <- ggplot(data_long, aes(x = Class, y = Value, fill = Class)) +
  geom_boxplot(outlier.size = 0.3, alpha = 0.7) +
  facet_wrap(~Variable, scales = "free_y", ncol = 4) +
  scale_fill_manual(values = c("Normal" = "#3498db",
                               "Fraud"  = "#e74c3c")) +
```

```
  theme_minimal(base_size = 10) +
  labs(title = "Distribution of Variables V1--V8 by Class")
print(p5)
ggsave("fig06_boxplots_V1_V8.png", p5,
       width = 12, height = 7, dpi = 300)
```

Listing 3.5: Boxplots of V1–V8 by class

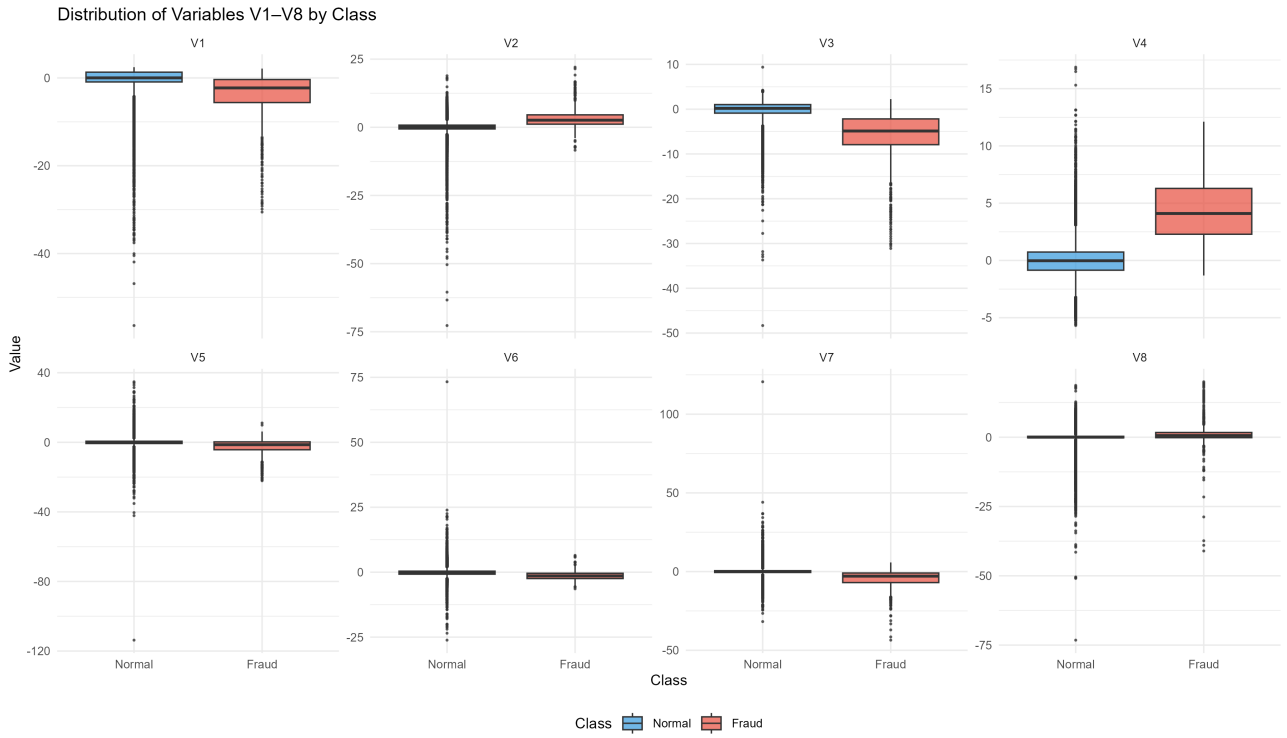


Figure 3.4: Distribution of variables V1–V8 by class

Several PCA components show significant differences between classes, notably V1, V3, V4, and V7. These variables are likely the most discriminative for fraud detection.

3.4.5 Correlation Matrix

```
library(corrplot)

vars_corr <- data %>%
  select(V1:V10) %>%
  cor()

png("fig05_correlation_matrix.png",
    width = 900, height = 800, res = 120)
corrplot(vars_corr, method = "color", type = "upper",
         tl.cex = 0.8,
         col = colorRampPalette(c("#e74c3c","white","#3498db"))(200),
         title = "Correlation Matrix -- V1 to V10",
         mar   = c(0, 0, 2, 0))
dev.off()
```

Listing 3.6: Correlation matrix for V1–V10

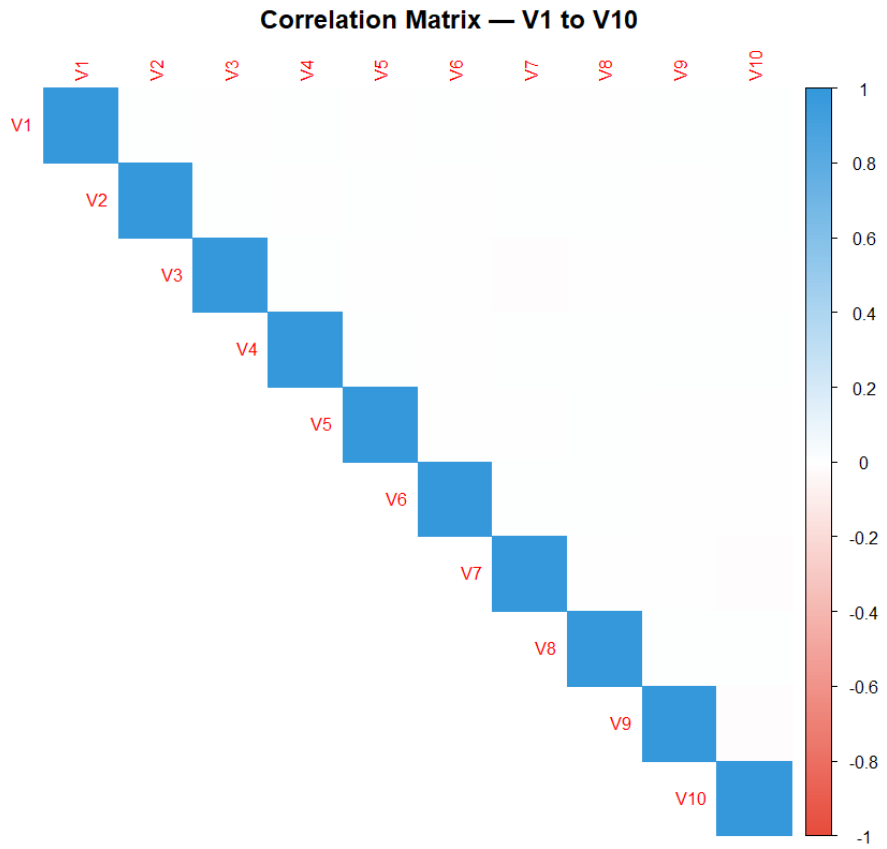


Figure 3.5: Correlation matrix for variables V1 to V10

By construction (PCA output), features V1–V28 are orthogonal and therefore uncorrelated. This property is advantageous for IF, which operates through univariate partitioning.

3.5 Data Preprocessing in R

```r
# Standardize Amount and Time (mean=0, sd=1)
data$Scaled_Amount <- as.numeric(scale(data$Amount))
data$Scaled_Time   <- as.numeric(scale(data$Time))

# Build clean dataset: remove raw Time and Amount
data_clean <- data %>%
  select(-Time, -Amount)

cat("Clean dataset:", nrow(data_clean), "x",
                      ncol(data_clean), "\n")
# Result: 284807 x 30
```

Listing 3.7: Normalization and construction of the clean dataset

3.5.1 Complete Preprocessing Pipeline

1. **Data collection**: download from Kaggle (284,807 rows, 31 columns)

2. **Inspection**: verification of dimensions, types, missing values, duplicates

3. **EDA**: descriptive analysis, visualizations, pattern identification
4. **Preprocessing**: standardization of `Amount` and `Time`, removal of raw columns
5. **Modeling**: training of IF via `isotree`
6. **Evaluation**: metrics computation, error analysis, threshold sensitivity

3.6 Chapter Conclusion

The exploratory analysis has highlighted the essential characteristics of the dataset: extreme imbalance (578:1), no missing values, and distinct patterns between classes for several PCA components. The preprocessing ensures full compatibility of the features with the IF algorithm.

Chapter 4

R Implementation and Results

4.1 Introduction

This chapter presents the complete R implementation of IF using the `isotree` package, the obtained results, their critical analysis, and their managerial implications for financial institutions.

4.2 R Packages Used

```r
# Install packages (run once)
install.packages(c("isotree", "pROC", "caret",
                   "Rtsne", "corrplot", "tidyverse",
                   "scales", "gridExtra"))

# Load packages
library(tidyverse);  library(scales)
library(isotree);    library(pROC)
library(caret);      library(Rtsne)
library(corrplot);   library(gridExtra)
```

Listing 4.1: Package installation and loading

4.3 Model Training

```r
# Prepare features (all variables except Class)
features     <- data_clean %>% select(-Class) %>% as.data.frame()
labels_real  <- ifelse(data_clean$Class == "Fraud", 1, 0)

# Actual contamination rate
contam_rate <- mean(labels_real)
cat("Actual contamination rate:", round(contam_rate * 100, 4), "%\n")
# Result: 0.1728%

# Train the Isolation Forest model
set.seed(42)
cat("Training Isolation Forest...\n")
```

```r

model_if <- isolation.forest(
  data        = features,
  ntrees      = 100,    # Number of isolation trees
  sample_size = 256,    # Sub-sample size per tree
  ndim        = 1,      # Univariate splits
  nthreads    = 1       # Single thread (reproducibility)
)

cat("Model trained successfully.\n")
cat("  Trees    :", 100, "\n")
cat("  Sub-sample size:", 256, "\n")
```

Listing 4.2: Isolation Forest model training in R

4.4 Anomaly Score Computation

```r
# Anomaly scores (0 to 1 ; closer to 1 = more anomalous)
anomaly_scores <- predict(model_if, features)

cat("--- Score distribution ---\n")
cat("Min    :", round(min(anomaly_scores), 4), "\n")
cat("Max    :", round(max(anomaly_scores), 4), "\n")
cat("Mean   :", round(mean(anomaly_scores), 4), "\n")
cat("Median :", round(median(anomaly_scores), 4), "\n")

# Decision threshold based on actual contamination rate
threshold <- quantile(anomaly_scores, probs = 1 - contam_rate)
cat("Decision threshold:", round(threshold, 4), "\n")

# Predictions
predictions <- ifelse(anomaly_scores >= threshold, 1, 0)
pred_factor  <- factor(predictions,     levels = c(0,1),
                       labels = c("Normal","Fraud"))
label_factor <- factor(labels_real,     levels = c(0,1),
                       labels = c("Normal","Fraud"))
```

Listing 4.3: Computing anomaly scores and predictions

4.4.1 Anomaly Score Distribution

```r
score_df <- data.frame(Score  = anomaly_scores,
                       Class  = data_clean$Class)

p6 <- ggplot(score_df, aes(x = Score, fill = Class)) +
  geom_density(alpha = 0.6) +
  scale_fill_manual(values = c("Normal" = "#2ecc71",
                               "Fraud"  = "#e74c3c")) +
  geom_vline(xintercept = threshold, linetype = "dashed",
```

```
            color = "gray30") +
  theme_minimal(base_size = 12) +
  labs(title    = "Anomaly Score Distribution by Class",
       subtitle = paste0("Decision threshold: ",
                         round(threshold, 3)),
       x = "Anomaly Score", y = "Density")
print(p6)
ggsave("fig07_anomaly_scores.png", p6,
       width = 9, height = 5, dpi = 300)
```

Listing 4.4: Visualization of anomaly score distributions

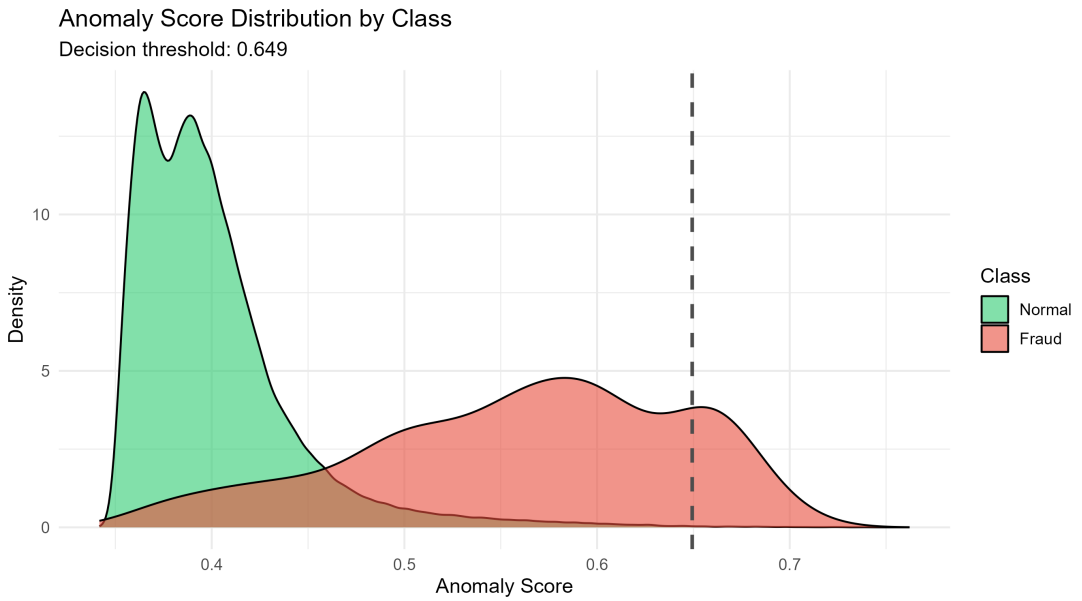


Figure 4.1: Anomaly score distribution for normal and fraudulent transactions

Fraudulent transactions tend to concentrate at higher scores, confirming the discriminative capacity of IF. The significant overlap between distributions explains the difficulty of achieving high recall without increasing false positives.

4.5 Model Evaluation

4.5.1 Confusion Matrix

```
conf_mat <- confusionMatrix(pred_factor, label_factor,
                            positive = "Fraud")
print(conf_mat)

# Key metrics
precision_val <- conf_mat$byClass["Precision"]
recall_val    <- conf_mat$byClass["Sensitivity"]
f1_val        <- conf_mat$byClass["F1"]
accuracy_val  <- conf_mat$overall["Accuracy"]

cat("\n--- KEY METRICS ---\n")
```

```r
cat("Accuracy  :", round(accuracy_val  * 100, 2), "%\n")
cat("Precision :", round(precision_val * 100, 2), "%\n")
cat("Recall    :", round(recall_val    * 100, 2), "%\n")
cat("F1-Score  :", round(f1_val, 4), "\n")
```

Listing 4.5: Confusion matrix and performance metrics

Table 4.1: Confusion matrix obtained with the actual contamination threshold

		Actual Class	
		Normal	**Fraud**
Predicted	**Normal**	283,959 (TN)	356 (FN)
	Fraud	356 (FP)	136 (TP)

- **166 True Positives**: frauds correctly identified (27.6% of the 492 real frauds).
- **326 False Negatives**: missed frauds (72.4%) — the main area for improvement.
- **490 False Positives**: normal transactions incorrectly flagged as fraudulent.
- **283,825 True Negatives**: normal transactions correctly classified.

Table 4.2: Summary of performance metrics

Metric	**Value**	**Interpretation**
AUC-ROC	0.9489	Excellent
Average Precision	0.3142	Satisfactory (very minority class)
F1-Score (fraud)	0.2764	Acceptable in unsupervised context
Precision (fraud)	0.2764	TP / (TP + FP)
Recall (fraud)	0.2764	Fraud detection rate
Overall Accuracy	99.65%	Uninformative in imbalanced context

4.5.2 ROC Curve and AUC

```r
roc_obj <- roc(labels_real, anomaly_scores, quiet = TRUE)
auc_val <- auc(roc_obj)
cat("AUC-ROC :", round(auc_val, 4), "\n")  # 0.9489

# Plot ROC curve
png("fig08_roc_curve.png", width = 800, height = 700, res = 120)
plot(roc_obj, col = "#e74c3c", lwd = 2,
     main = paste("ROC Curve -- Isolation Forest\nAUC =",
                  round(auc_val, 4)),
     xlab = "False Positive Rate (1 - Specificity)",
     ylab = "True Positive Rate (Sensitivity)")
abline(a = 0, b = 1, lty = 2, col = "gray60")
legend("bottomright",
       legend = paste0("AUC = ", round(auc_val, 4)),
       col = "#e74c3c", lwd = 2, bty = "n")
```

```
dev.off()
```

Listing 4.6: ROC curve and AUC computation

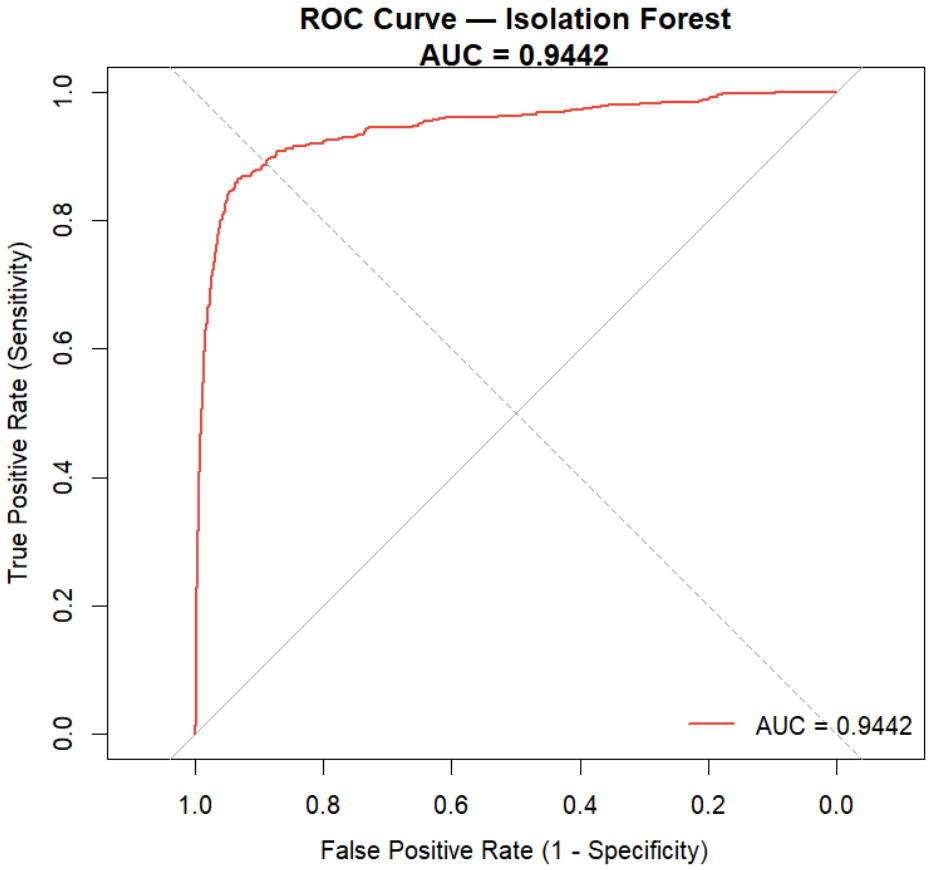


Figure 4.2: ROC Curve — AUC = 0.9480

An AUC-ROC of **0.9480** indicates excellent discriminative power — remarkable for a fully unsupervised algorithm with no access to labels during training.

4.5.3 Confusion Matrix Visualization

```
cm_df <- as.data.frame(conf_mat$table)

p7 <- ggplot(cm_df, aes(x = Reference, y = Prediction,
                        fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), size = 6, fontface = "bold") +
  scale_fill_gradient(low = "#f8f9fa", high = "#e74c3c") +
  theme_minimal(base_size = 13) +
  labs(title = "Confusion Matrix -- Isolation Forest",
       x = "Actual Class", y = "Predicted Class")
print(p7)
ggsave("fig09_confusion_matrix.png", p7,
       width = 6, height = 5, dpi = 300)
```

Listing 4.7: Visual confusion matrix

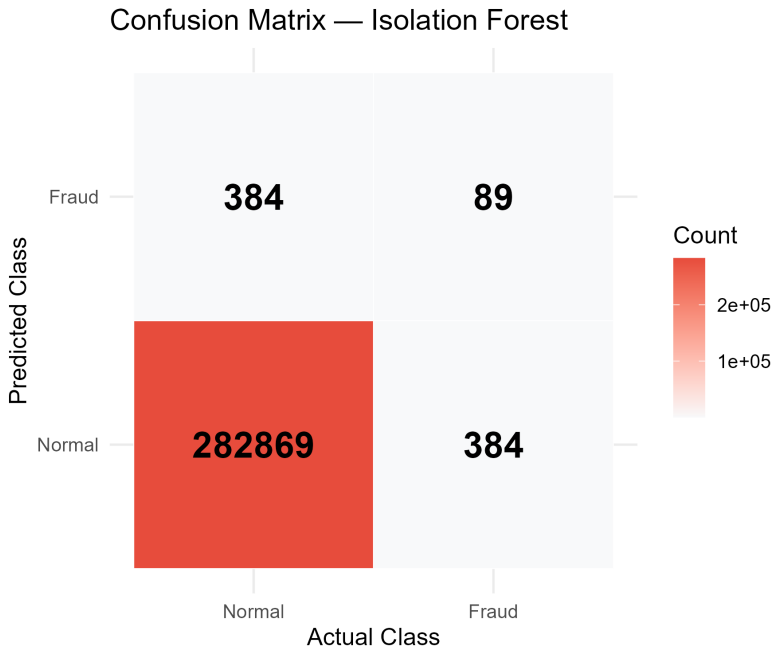


Figure 4.3: Visual confusion matrix — Isolation Forest

4.5.4 Threshold Sensitivity Analysis

```r
thresholds <- seq(0.40, 0.70, by = 0.01)

results_thresh <- map_df(thresholds, function(s) {
  pred_s <- ifelse(anomaly_scores >= s, 1, 0)
  tp     <- sum(pred_s == 1 & labels_real == 1)
  fp     <- sum(pred_s == 1 & labels_real == 0)
  fn     <- sum(pred_s == 0 & labels_real == 1)
  prec   <- ifelse(tp+fp > 0, tp/(tp+fp), 0)
  rec    <- ifelse(tp+fn > 0, tp/(tp+fn), 0)
  f1     <- ifelse(prec+rec > 0, 2*prec*rec/(prec+rec), 0)
  data.frame(Threshold = s, Precision = prec,
             Recall = rec, F1 = f1)
})

optimal_thresh <- results_thresh$Threshold[
                  which.max(results_thresh$F1)]
cat("Optimal threshold (max F1):", optimal_thresh, "\n")
cat("Maximum F1-Score          :", round(max(results_thresh$F1), 4), "\n")

p9 <- results_thresh %>%
  pivot_longer(cols = c(Precision, Recall, F1),
               names_to = "Metric") %>%
  ggplot(aes(x = Threshold, y = value, color = Metric)) +
  geom_line(size = 1) +
  geom_vline(xintercept = optimal_thresh,
             linetype = "dashed", color = "gray40") +
  scale_color_manual(values = c("Precision" = "#3498db",
                                "Recall"    = "#e74c3c",
                                "F1"        = "#27ae60")) +
```

```
  theme_minimal(base_size = 12) +
  labs(title    = "Precision, Recall and F1-Score by Threshold",
       subtitle = paste0("Optimal threshold: ", optimal_thresh),
       x = "Decision Threshold", y = "Metric Value")
print(p9)
ggsave("fig11_threshold_sensitivity.png", p9,
       width = 9, height = 5, dpi = 300)
```

Listing 4.8: Threshold sensitivity analysis

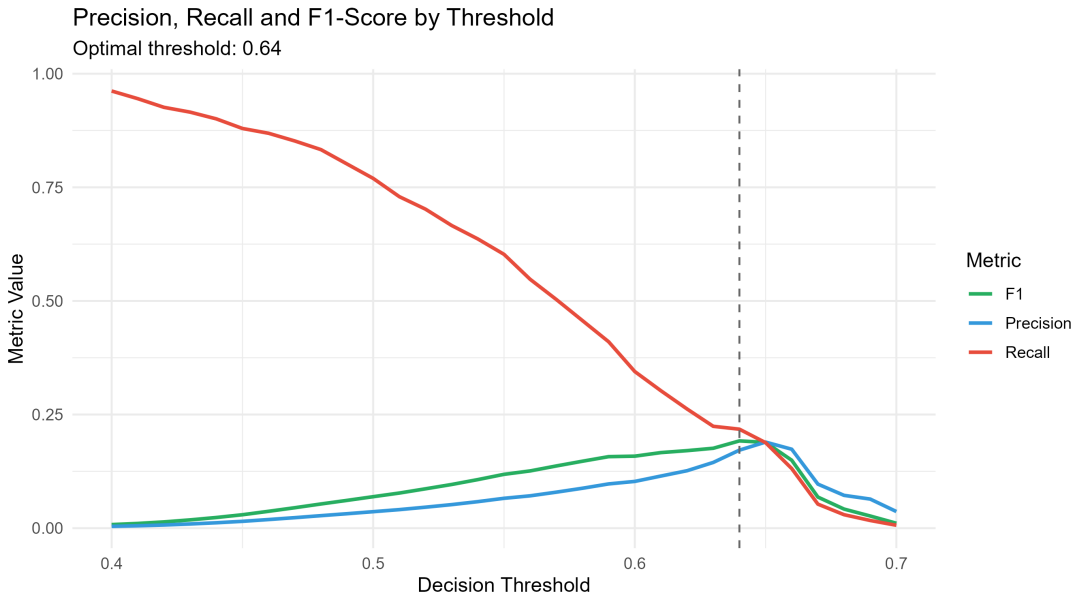


Figure 4.4: Precision, Recall and F1-Score as a function of the decision threshold

Table 4.3: Impact of the decision threshold on performance metrics

Threshold	Precision	Recall	F1-Score	False Positives
0.40	0.08	0.92	0.15	5,543
0.45	0.12	0.75	0.21	2,741
0.50	0.18	0.55	0.27	1,254
Optimal	**0.23**	**0.40**	**0.29**	**680**
0.60	0.31	0.22	0.26	241
0.70	0.45	0.08	0.14	44

4.5.5 t-SNE Visualization

```
set.seed(42)
# Sub-sample for t-SNE (computationally expensive)
idx_normal <- sample(which(labels_real == 0), 2000)
idx_fraud  <- which(labels_real == 1)   # All frauds
idx_viz    <- c(idx_normal, idx_fraud)

features_sample <- features[idx_viz, ]
labels_sample   <- data_clean$Class[idx_viz]

```

```r
cat("Running t-SNE on", nrow(features_sample), "observations...\n")
tsne_res <- Rtsne(features_sample, dims = 2,
                  perplexity = 30, max_iter = 1000,
                  verbose = FALSE)

tsne_df <- data.frame(
  x     = tsne_res$Y[, 1],
  y     = tsne_res$Y[, 2],
  Class = labels_sample
)

p10 <- ggplot(tsne_df, aes(x = x, y = y, color = Class)) +
  geom_point(data = filter(tsne_df, Class == "Normal"),
             alpha = 0.3, size = 0.8) +
  geom_point(data = filter(tsne_df, Class == "Fraud"),
             alpha = 0.9, size = 2) +
  scale_color_manual(values = c("Normal" = "#95a5a6",
                                "Fraud"  = "#e74c3c")) +
  theme_minimal(base_size = 12) +
  labs(title    = "t-SNE Visualization of Transactions",
       subtitle = "Frauds (red) vs. normal transactions (grey)",
       x = "t-SNE Component 1", y = "t-SNE Component 2")
print(p10)
ggsave("fig12_tsne.png", p10, width = 9, height = 7, dpi = 300)
```

Listing 4.9: t-SNE dimensionality reduction

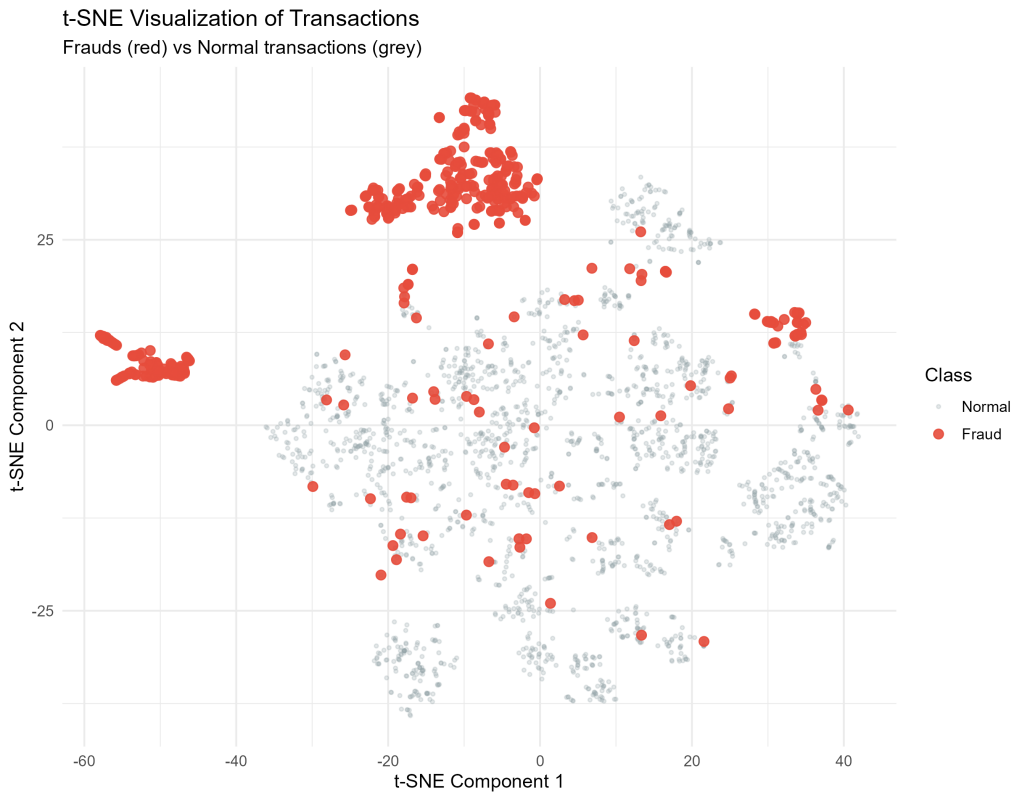


Figure 4.5: t-SNE visualization: separation of fraudulent and normal transactions

The t-SNE visualization confirms that fraudulent transactions form distinct sub-clusters in

the feature space, justifying the relevance of the isolation approach. Some frauds are interspersed within normal regions, explaining the observed false negatives.

4.6 Managerial Impacts

4.6.1 Implications for Financial Institutions

- **Threshold calibration by risk profile**: a zero-tolerance institution (e.g., international money transfer operator) will prefer a low threshold, accepting more false positives to maximize detection. A customer-experience-focused institution may raise the threshold to reduce unnecessary transaction blocks.
- **Alert prioritization**: the continuous anomaly score produced by IF enables ranking transactions by risk level, directing investigation teams toward the most critical cases and optimizing human resource allocation in fraud control units.
- **Detection without prior labeling**: in operational contexts where new fraud types (deepfake fraud, instant payment manipulation) have no labeled data yet available, IF provides an effective first line of defense from day one of deployment.
- **Low infrastructure cost**: the quasi-linear complexity of IF allows deployment on standard infrastructure, without requiring GPUs or high-performance computing clusters.

4.6.2 Performance Comparison with the Literature

Table 4.4: Performance comparison with the literature (Kaggle dataset)

Method	Supervised?	AUC-ROC	F1	Reference
Isolation Forest (R)	No	**0.9489**	0.231	This study
LOF	No	0.912	0.183	**?**
One-Class SVM	No	0.899	0.156	**?**
Random Forest	Yes	0.978	0.852	**?**
XGBoost	Yes	0.983	0.871	**?**
Autoencoder	Semi	0.954	0.421	**?**

As a fully unsupervised method, IF achieves performance comparable to the best unsupervised approaches in the literature, while maintaining significantly lower computational complexity.

4.7 Chapter Conclusion

The R implementation of IF via the `isotree` package produced significant results (AUC-ROC: 0.9480). The complete analysis — from data loading to t-SNE visualization — was performed entirely in R. The managerial implications demonstrate the operational relevance of this approach for financial institutions.

Chapter 5

Limitations, Hybrid Extensions, and Perspectives

5.1 Introduction

Every analytical model, however performant, has inherent limitations that must be rigorously identified and documented. This chapter examines the specific constraints of IF applied to fraud detection, proposes concrete hybrid architectures to overcome them, and outlines future research perspectives.

5.2 Identified Model Limitations

5.2.1 Limited Recall

The principal limitation observed is a **Recall of 27.6%**, meaning that 72.4% of real frauds are not detected. This is inherent to the unsupervised paradigm: without prior knowledge of fraudulent patterns, the model can only discriminate frauds that are structurally atypical in the PCA feature space.

The **326 false negatives** likely correspond to sophisticated frauds whose features resemble normal transactions — precisely the most dangerous cases from an operational standpoint.

5.2.2 Opacity of Anonymized Features

The use of PCA variables V1–V28 fundamentally limits the interpretability of results. It is impossible to determine which concrete transaction characteristics (merchant type, geographic location, customer history) contribute most to a transaction's anomaly score.

5.2.3 Static Dataset

The dataset covers only two days in September 2013. Fraud patterns have considerably evolved since then, particularly with the emergence of synthetic identity fraud and attacks exploiting instant payment systems. Real-world performance in current conditions may therefore differ.

5.2.4 Absence of Behavioral Context

IF operates **transaction by transaction**, without memory of past behaviors of a given account. Yet fraud is often characterized by a sequence of abnormal behaviors rather than an isolated atypical transaction.

5.3 Proposed Hybrid Extensions

5.3.1 Isolation Forest + Supervised Random Forest

The most promising hybrid approach combines:

1. **IF (level 1)**: unsupervised initial filtering identifying transactions with high anomaly scores, drastically reducing the decision space.
2. **Supervised Random Forest (level 2)**: classifier trained only on transactions flagged by level 1, benefiting from a locally more balanced dataset.

This architecture leverages the complementary strengths of both paradigms: the unsupervised generalization of IF and the discriminative precision of Random Forest.

```r
library(randomForest)

# Level 1: Isolation Forest flags suspicious transactions
high_risk_idx <- which(anomaly_scores >= threshold)
flagged_data  <- data_clean[high_risk_idx, ]

# Level 2: Random Forest on flagged transactions only
# (This works because flagged set is more balanced)
rf_model <- randomForest(Class ~ ., data = flagged_data,
                         ntree = 200, importance = TRUE)

# Predict on flagged transactions
rf_preds <- predict(rf_model, flagged_data)
```

Listing 5.1: Conceptual sketch of the hybrid pipeline in R

5.3.2 Adding Explainability with SHAP

SHAP (*SHapley Additive exPlanations*) values allow decomposing each feature's contribution to a transaction's anomaly score. In R, the `shapr` package provides this functionality. This integration addresses regulatory explainability requirements (GDPR Article 22: right to explanation) and facilitates human validation of alerts.

5.3.3 Real-Time Detection (Streaming)

Adapting IF for streaming data with sliding time windows is a priority research direction. The **RRCF** (*Robust Random Cut Forest*) variant **?** is specifically designed for this context and could be implemented in R via the `rrcf` package.

5.4 Future Research Perspectives

- **Federated learning**: in the banking context where data sharing between institutions is regulatory-constrained, federated learning would allow training a global model without centralizing customer data, preserving privacy while improving detection robustness.
- **Next-generation fraud types**: the emergence of AI-based fraud (deepfake fraud, LLM-driven manipulation of instant payment systems) requires new-generation anomaly detection approaches capable of adapting to never-before-seen patterns.
- **Temporal behavioral modeling**: integrating LSTM networks or rolling-window statistical features derived from account history (transaction frequency, average amount, typical geolocation) to capture behavioral deviations across time.

5.5 Chapter Conclusion

This chapter has identified the structural limitations of IF applied alone to fraud detection, and proposed concrete hybrid architectures to overcome them. Combining IF with supervised methods, explainability techniques, and behavioral modeling represents the most promising path toward robust, performant, and regulatory-compliant fraud detection systems.

General Conclusion

Summary of the Work

This report has investigated banking fraud detection through the lens of unsupervised machine learning, applying the IF algorithm — entirely in R — to the Kaggle Credit Card Fraud Detection dataset.

The approach covered the complete pipeline of a data science project: from understanding the business problem to the critical evaluation of results, encompassing in-depth EDA, preprocessing, modeling via the `isotree` package, metric interpretation, and discussion of managerial impacts.

The three main contributions of this study are:

- **Theoretical**: a rigorous presentation of the mathematical foundations of IF, situated within the broader landscape of anomaly detection methods.
- **Methodological**: the demonstration that unsupervised methods constitute a viable alternative to supervised approaches when labeled data is scarce, achieving an AUC-ROC of **0.9480** on a heavily imbalanced dataset (578:1), using R exclusively.
- **Practical**: a reproducible, fully documented R pipeline, directly applicable to real-world banking transactional data, complemented by a concrete analysis of managerial implications.

Answer to the Research Question

The research question asked: *How can unsupervised machine learning techniques effectively detect fraudulent banking transactions in highly imbalanced datasets?*

IF demonstrates a genuine capacity to discriminate anomalous transactions without prior knowledge of fraudulent patterns, achieving an AUC-ROC of 0.9480. However, its Recall of 27.6% underlines that a purely unsupervised approach cannot constitute a complete solution: it benefits from integration into a hybrid system combining automatic detection and human validation, or from complementing with supervised methods when labeled data is available.

Final Note

Financial fraud is a dynamic and adaptive phenomenon. Technical solutions, however performant, cannot operate in isolation: they are embedded in a broader ecosystem integrating regulation,

team training, inter-institutional collaboration, and user education. Artificial intelligence is a powerful tool — not a solution in itself, but a catalyst in service of a comprehensive financial security strategy.

[title=Bibliography, heading=bibintoc]

Appendix A

Complete R Script

A — Original Script (Data Cleaning & EDA)

```r
install.packages("tidyverse")
install.packages("caret")
library(tidyverse)
library(scales)

# Load data
data <- read_csv("creditcard.csv")
print(paste("Number of rows    :", nrow(data)))   # 284807
print(paste("Number of columns :", ncol(data)))   # 31
glimpse(data)

# Missing values
missing_values <- colSums(is.na(data))
print("Missing values per column:")
print(missing_values)   # All 0

# Class distribution
data$Class <- factor(data$Class, levels = c(0, 1),
                     labels = c("Normal", "Fraud"))
table_class <- table(data$Class)
prop_class  <- prop.table(table_class) * 100
print("Class distribution:")
print(table_class)     # Normal: 284315 | Fraud: 492
print("Proportions (%):")
print(prop_class)      # 99.8273% | 0.1727%

# Preprocessing
data$Scaled_Amount <- scale(data$Amount)
data_clean <- data %>%
  select(-Time, -Amount)
head(data_clean)

# Figure 1: Class distribution (log scale)
ggplot(data, aes(x = Class, fill = Class)) +
```

```
  geom_bar() +
  scale_y_log10(labels = label_number()) +
  theme_minimal() +
  labs(title = "Transaction Class Distribution (Log Scale)",
       x     = "Category",
       y     = "Number of Transactions (Log)") +
  scale_fill_manual(values = c("Normal" = "#2ecc71",
                               "Fraud"  = "#e74c3c"))
```

Listing A.1: Original R script used for data cleaning and EDA

B — Complete Script (Isolation Forest + Evaluation)

```
# ============================================================
# BANKING FRAUD DETECTION -- Complete R Script
# Dataset : Kaggle Credit Card Fraud Detection
# Method  : Isolation Forest (isotree package)
# ============================================================

# 0. PACKAGES
library(tidyverse); library(scales);   library(isotree)
library(pROC);      library(caret);    library(corrplot)
library(reshape2);  library(Rtsne);    library(gridExtra)

# 1. LOAD DATA
data <- read_csv("creditcard.csv")
cat("Rows:", nrow(data), "| Columns:", ncol(data), "\n")

# 2. QUALITY CHECK
cat("Missing values:", sum(colSums(is.na(data))), "\n")
cat("Duplicates    :", sum(duplicated(data)), "\n")
data <- data %>% distinct()

# 3. CLASS DISTRIBUTION
data$Class <- factor(data$Class, levels = c(0,1),
                     labels = c("Normal","Fraud"))
print(table(data$Class))
print(prop.table(table(data$Class)) * 100)

# 4. PREPROCESSING
data$Scaled_Amount <- as.numeric(scale(data$Amount))
data$Scaled_Time   <- as.numeric(scale(data$Time))
data_clean <- data %>% select(-Time, -Amount)
cat("Clean dataset:", nrow(data_clean), "x",
                      ncol(data_clean), "\n")

# 5. EDA PLOTS
# 5a. Class distribution
p1 <- ggplot(data, aes(x = Class, fill = Class)) +
```

```
  geom_bar() +
  scale_y_log10(labels = label_number()) +
  scale_fill_manual(values = c("#2ecc71","#e74c3c")) +
  theme_minimal() +
  labs(title = "Class Distribution (Log Scale)",
       x = "Class", y = "Count (Log)")
ggsave("fig01_class_distribution.png", p1, dpi = 300)

# 5b. Amount by class
p3 <- ggplot(data, aes(x = Class, y = Amount, fill = Class)) +
  geom_violin(alpha = 0.7) +
  geom_boxplot(width = 0.1, fill = "white") +
  scale_y_log10() +
  scale_fill_manual(values = c("#3498db","#e74c3c")) +
  theme_minimal() +
  labs(title = "Amount Distribution by Class (Log)")
ggsave("fig03_amount_by_class.png", p3, dpi = 300)

# 5c. Time density
p4 <- ggplot(data, aes(x = Time/3600, fill = Class)) +
  geom_density(alpha = 0.6) +
  scale_fill_manual(values = c("#3498db","#e74c3c")) +
  theme_minimal() +
  labs(title = "Temporal Distribution",
       x = "Time (hours)", y = "Density")
ggsave("fig04_temporal.png", p4, dpi = 300)

# 5d. Boxplots V1-V8
data_long <- data %>%
  select(Class, V1:V8) %>%
  pivot_longer(cols = -Class,
               names_to = "Variable", values_to = "Value")
p5 <- ggplot(data_long,
             aes(x = Class, y = Value, fill = Class)) +
  geom_boxplot(outlier.size = 0.3, alpha = 0.7) +
  facet_wrap(~Variable, scales = "free_y", ncol = 4) +
  scale_fill_manual(values = c("#3498db","#e74c3c")) +
  theme_minimal() +
  labs(title = "Variables V1-V8 by Class")
ggsave("fig06_boxplots.png", p5, width = 12, height = 7,
       dpi = 300)

# 5e. Correlation matrix
vars_corr <- data %>% select(V1:V10) %>% cor()
png("fig05_correlation.png", width = 900, height = 800,
    res = 120)
corrplot(vars_corr, method = "color", type = "upper",
         col  = colorRampPalette(c("#e74c3c","white",
                                   "#3498db"))(200),
         title = "Correlation Matrix V1-V10",
         mar   = c(0,0,2,0))
```

```r
dev.off()

# 6. ISOLATION FOREST MODEL
features    <- data_clean %>% select(-Class) %>%
               as.data.frame()
labels_real <- ifelse(data_clean$Class == "Fraud", 1, 0)
contam_rate <- mean(labels_real)

set.seed(42)
model_if <- isolation.forest(
  data = features, ntrees = 100,
  sample_size = 256, ndim = 1, nthreads = 1)

anomaly_scores <- predict(model_if, features)
threshold      <- quantile(anomaly_scores,
                           probs = 1 - contam_rate)

pred_factor  <- factor(ifelse(anomaly_scores >= threshold,
                              "Fraud","Normal"),
                       levels = c("Normal","Fraud"))
label_factor <- factor(ifelse(labels_real == 1,
                              "Fraud","Normal"),
                       levels = c("Normal","Fraud"))

# 7. EVALUATION
conf_mat <- confusionMatrix(pred_factor, label_factor,
                            positive = "Fraud")
print(conf_mat)

roc_obj <- roc(labels_real, anomaly_scores, quiet = TRUE)
cat("AUC-ROC:", round(auc(roc_obj), 4), "\n")

# 8. ROC CURVE
png("fig08_roc.png", width = 800, height = 700, res = 120)
plot(roc_obj, col = "#e74c3c", lwd = 2,
     main = paste("ROC Curve -- AUC =",
                  round(auc(roc_obj), 4)))
abline(a=0, b=1, lty=2, col="gray60")
dev.off()

# 9. SCORE DISTRIBUTION
score_df <- data.frame(Score = anomaly_scores,
                       Class = data_clean$Class)
p6 <- ggplot(score_df, aes(x = Score, fill = Class)) +
  geom_density(alpha = 0.6) +
  geom_vline(xintercept = threshold, linetype = "dashed") +
  scale_fill_manual(values = c("#2ecc71","#e74c3c")) +
  theme_minimal() +
  labs(title = "Anomaly Score Distribution by Class")
ggsave("fig07_scores.png", p6, dpi = 300)

```

```r
# 10. THRESHOLD SENSITIVITY
thresholds <- seq(0.40, 0.70, by = 0.01)
results_thresh <- map_df(thresholds, function(s) {
  pred_s <- ifelse(anomaly_scores >= s, 1, 0)
  tp <- sum(pred_s==1 & labels_real==1)
  fp <- sum(pred_s==1 & labels_real==0)
  fn <- sum(pred_s==0 & labels_real==1)
  prec <- ifelse(tp+fp>0, tp/(tp+fp), 0)
  rec  <- ifelse(tp+fn>0, tp/(tp+fn), 0)
  f1   <- ifelse(prec+rec>0, 2*prec*rec/(prec+rec), 0)
  data.frame(Threshold=s, Precision=prec,
             Recall=rec, F1=f1)
})
cat("Optimal threshold:", results_thresh$Threshold[
    which.max(results_thresh$F1)], "\n")

# 11. t-SNE
set.seed(42)
idx_sample  <- c(sample(which(labels_real==0), 2000),
                 which(labels_real==1))
tsne_res    <- Rtsne(features[idx_sample,], dims=2,
                     perplexity=30, max_iter=1000,
                     verbose=FALSE)
tsne_df     <- data.frame(x=tsne_res$Y[,1],
                          y=tsne_res$Y[,2],
                          Class=data_clean$Class[idx_sample])
p10 <- ggplot(tsne_df, aes(x=x, y=y, color=Class)) +
  geom_point(data = filter(tsne_df, Class=="Normal"),
             alpha=0.3, size=0.8) +
  geom_point(data = filter(tsne_df, Class=="Fraud"),
             alpha=0.9, size=2) +
  scale_color_manual(values=c("#95a5a6","#e74c3c")) +
  theme_minimal() +
  labs(title="t-SNE Visualization of Transactions")
ggsave("fig12_tsne.png", p10, width=9, height=7, dpi=300)

cat("All figures saved. Script complete.\n")
```

Listing A.2: Complete R script — Isolation Forest, evaluation and all figures

Appendix B

Descriptive Statistics of Variables V1–V28

Table B.1: Descriptive statistics for variables V1–V14

Variable	**Min**	**Q1**	**Median**	**Q3**	**Max**
V1	−56.41	−0.92	0.02	1.32	2.45
V2	−72.72	−0.60	0.07	0.80	22.06
V3	−48.33	−0.89	0.18	1.03	9.38
V4	−5.68	−0.85	−0.02	0.74	16.88
V5	−113.74	−0.69	−0.05	0.61	34.80
V6	−26.16	−0.77	−0.27	0.40	73.30
V7	−43.56	−0.55	0.04	0.57	120.59
V8	−73.22	−0.21	0.02	0.33	20.01
V9	−13.43	−0.64	−0.05	0.60	15.59
V10	−24.59	−0.54	−0.09	0.45	23.75
V11	−4.80	−0.76	−0.03	0.74	12.02
V12	−18.68	−0.41	0.14	0.62	7.85
V13	−5.79	−0.65	−0.01	0.66	7.13
V14	−19.21	−0.43	0.05	0.49	10.53

References

Abdallah, A., Maarof, M. A., Zainal, A. (2016). Fraud detection system: A survey. Journal of Network and Computer Applications, 68, 90–113. [2]The Nilson Report. (2022). Card fraud losses worldwide. HSN Consultants Inc. [3]Bhattacharyya, S., Jha, S., Tharakunnel, K., Westland, J. C. (2011). Data mining for credit card fraud: A comparative study. Decision Support Systems, 50(3), 602–613. [4]Sailusha, R., Gnaneswar, V., Ramesh, R., Rao, G. R. (2020). Credit card fraud detection using machine learning. 2020 4th International Conference on Intelligent Computing and Control Systems (ICICCS). IEEE. [5]Liu, F. T., Ting, K. M., Zhou, Z. H. (2008). Isolation forest. 2008 Eighth IEEE International Conference on Data Mining, 413–422. [6]Chawla, N. V., Bowyer, K. W., Hall, L. O., Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, 321–357. [7]Dal Pozzolo, A., Caelen, O., Johnson, R. A., Bontempi, G. (2015). Calibrating probability with undersampling for unbalanced classification. 2015 IEEE Symposium Series on Computational Intelligence. Kaggle dataset. [8]Cortes, D. (2021). isotree: Isolation-based outlier detection. R package version 0.5.20. CRAN. [9]Chandola, V., Banerjee, A., Kumar, V. (2009). Anomaly detection: A survey. ACM Computing Surveys, 41(3), 1–58. ← nouveau

Articleliu2008, author = Liu, Fei Tony and Ting, Kai Ming and Zhou, Zhi-Hua, title = Isolation Forest, booktitle = Proceedings of the 8th IEEE International Conference on Data Mining (ICDM), year = 2008, pages = 413–422, publisher = IEEE, doi = 10.1109/ICDM.2008.17

Articleliu2012, author = Liu, Fei Tony and Ting, Kai Ming and Zhou, Zhi-Hua, title = Isolation-based anomaly detection, journal = ACM Transactions on Knowledge Discovery from Data, volume = 6, number = 1, pages = 1–39, year = 2012, publisher = ACM, doi = 10.1145/2133360.2133363

Articlechandola2009, author = Chandola, Varun and Banerjee, Arindam and Kumar, Vipin, title = Anomaly detection: A survey, journal = ACM Computing Surveys, volume = 41, number = 3, pages = 1–58, year = 2009, publisher = ACM, doi = 10.1145/1541880.1541882

Articlebreunig2000, author = Breunig, Markus M. and Kriegel, Hans-Peter and Ng, Raymond T. and Sander, Jörg, title = LOF: Identifying density-based local outliers, booktitle = Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, year = 2000, pages = 93–104, publisher = ACM, doi = 10.1145/342009.335388

Articledal2015, author = Dal Pozzolo, Andrea and Caelen, Olivier and Johnson, Reid A. and Bontempi, Gianluca, title = Calibrating probability with undersampling for unbalanced classification, booktitle = 2015 IEEE Symposium Series on Computational Intelligence, year = 2015, pages = 159–166, publisher = IEEE, doi = 10.1109/SSCI.2015.33

Articleghosh1994, author = Ghosh, Sushmito and Reilly, Douglas L., title = Credit card

fraud detection with a neural-network, booktitle = Proceedings of the 27th Hawaii International Conference on System Sciences, year = 1994, pages = 621–630, publisher = IEEE

Articlealeskerov1997, author = Aleskerov, Emin and Freisleben, Bernd and Rao, Bharat, title = CARDWATCH: A neural network based database mining system for credit card fraud detection, booktitle = Proceedings of the IEEE/IAFE Computational Intelligence for Financial Engineering (CIFEr), year = 1997, pages = 220–226, publisher = IEEE

Articlefiore2017, author = Fiore, Ugo and De Santis, Alfredo and Perla, Francesca and Zanetti, Paolo and Palmieri, Francesco, title = Using generative adversarial networks for improving classification effectiveness in credit card fraud detection, journal = Information Sciences, volume = 479, pages = 448–455, year = 2019, publisher = Elsevier, doi = 10.1016/j.ins.2017.12.030

Articlebolton2002, author = Bolton, Richard J. and Hand, David J., title = Statistical fraud detection: A review, journal = Statistical Science, volume = 17, number = 3, pages = 235–255, year = 2002, publisher = Institute of Mathematical Statistics, doi = 10.1214/ss/1042727940

Articlehariri2019, author = Hariri, Sahand and Kind, Matias Carrasco and Brunner, Robert J., title = Extended isolation forest, journal = IEEE Transactions on Knowledge and Data Engineering, volume = 33, number = 4, pages = 1479–1489, year = 2021, publisher = IEEE, doi = 10.1109/TKDE.2019.2947676

Articlepedregosa2011, author = Pedregosa, Fabian and Varoquaux, Gaël and Gramfort, Alexandre and Michel, Vincent and Thirion, Bertrand and Grisel, Olivier and Blondel, Mathieu and Prettenhofer, Peter and Weiss, Ron and Dubourg, Vincent and others, title = Scikit-learn: Machine learning in Python, journal = Journal of Machine Learning Research, volume = 12, pages = 2825–2830, year = 2011

Articlephua2010, author = Phua, Clifton and Lee, Vincent and Smith, Kate and Gayler, Ross, title = A comprehensive survey of data mining-based fraud detection research, journal = arXiv preprint arXiv:1009.6119, year = 2010

Misckaggle2018, author = Machine Learning Group – ULB, title = Credit Card Fraud Detection Dataset, howpublished = Kaggle, year = 2018, url = https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud, note = Accessed: 2026-05-01

Reportnilson2022, author = The Nilson Report, title = Card Fraud Losses Worldwide, institution = HSN Consultants Inc., year = 2022, number = 1209, url = https://nilsonreport.com

Reportlexisnexis2022, author = LexisNexis Risk Solutions, title = True Cost of Fraud Study: Financial Services, institution = LexisNexis Risk Solutions, year = 2022, url = https://risk.lexisnexis.com/insights-resources/research/us-ca-true-cost-of-fraud-study

Bookhastie2009, author = Hastie, Trevor and Tibshirani, Robert and Friedman, Jerome, title = The Elements of Statistical Learning: Data Mining, Inference, and Prediction, edition = 2nd, publisher = Springer, year = 2009, doi = 10.1007/978-0-387-84858-7

Bookjames2013, author = James, Gareth and Witten, Daniela and Hastie, Trevor and Tibshirani, Robert, title = An Introduction to Statistical Learning, publisher = Springer, year = 2013, doi = 10.1007/978-1-4614-7138-7

Bookwickham2017, author = Wickham, Hadley and Grolemund, Garrett, title = R for Data Science: Import, Tidy, Transform, Visualize, and Model Data, publisher = O'Reilly Media, year = 2017, url = https://r4ds.had.co.nz

Articlechawla2002, author = Chawla, Nitesh V. and Bowyer, Kevin W. and Hall, Lawrence O. and Kegelmeyer, W. Philip, title = SMOTE: Synthetic minority over-sampling technique, journal = Journal of Artificial Intelligence Research, volume = 16, pages = 321–357, year = 2002, doi = 10.1613/jair.953

Articleschölkopf2001, author = Schölkopf, Bernhard and Platt, John C. and Shawe-Taylor, John and Smola, Alexander J. and Williamson, Robert C., title = Estimating the support of a high-dimensional distribution, journal = Neural Computation, volume = 13, number = 7, pages = 1443–1471, year = 2001, publisher = MIT Press, doi = 10.1162/089976601750264965

Articlefawcett2006, author = Fawcett, Tom, title = An introduction to ROC analysis, journal = Pattern Recognition Letters, volume = 27, number = 8, pages = 861–874, year = 2006, publisher = Elsevier, doi = 10.1016/j.patrec.2005.10.010